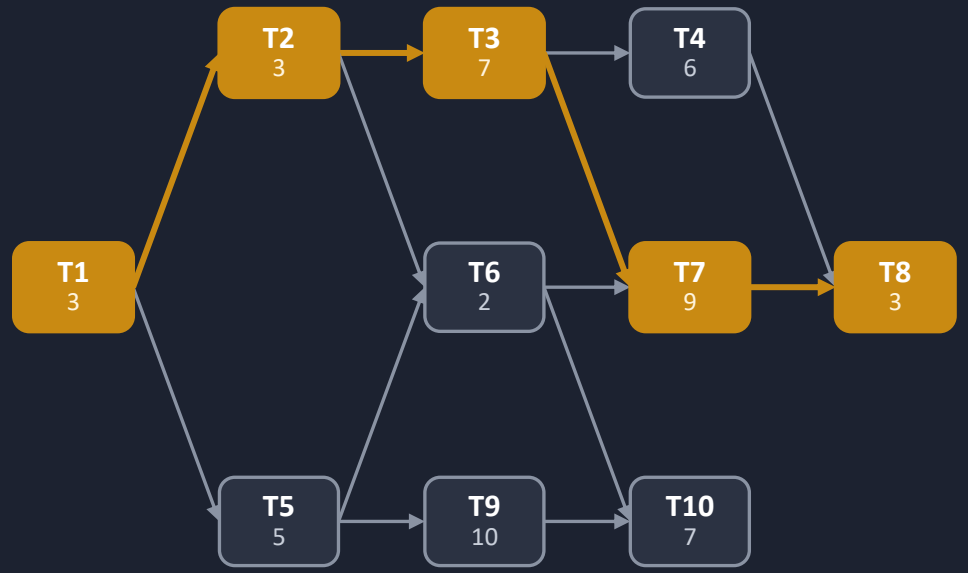
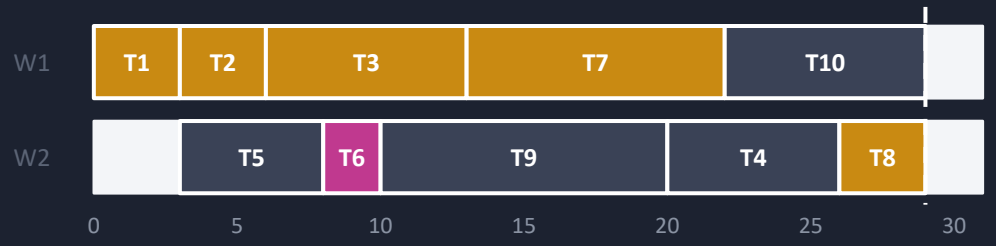
Adaptive Task Scheduler

Heuristic Scheduling, Lookahead and Experimental Weight Optimization

Research presentation · P | prec | Cmax · synthetic DAG benchmarks



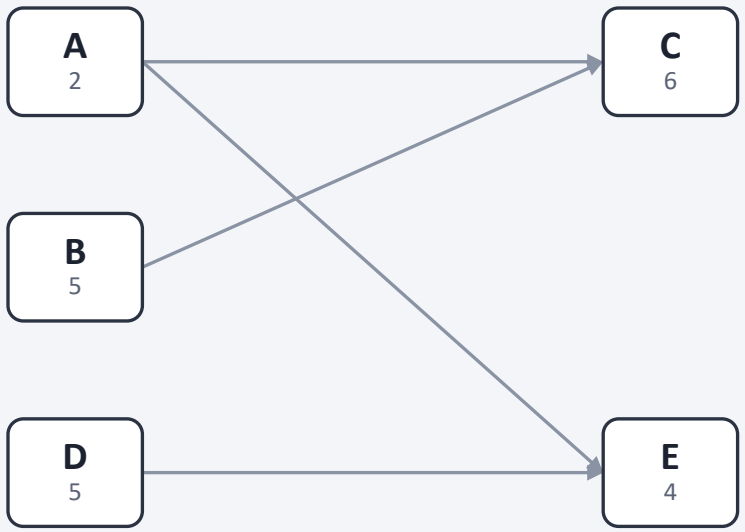


A real benchmark DAG (Phase 3, STENCIL-SMALL-81002-191) and the schedule our final scheduler builds for it: Cmax = 29, the proven optimum.

The problem: who works on what, and when?

Identical workers, tasks with known durations, and dependencies between them. Finish everything as early as possible.

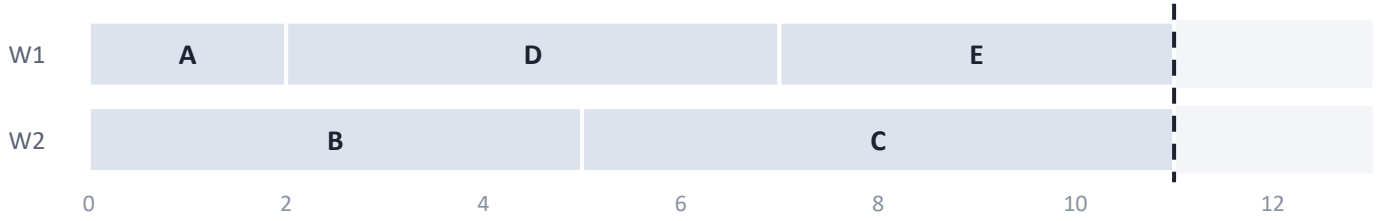
TASKS AND DEPENDENCIES



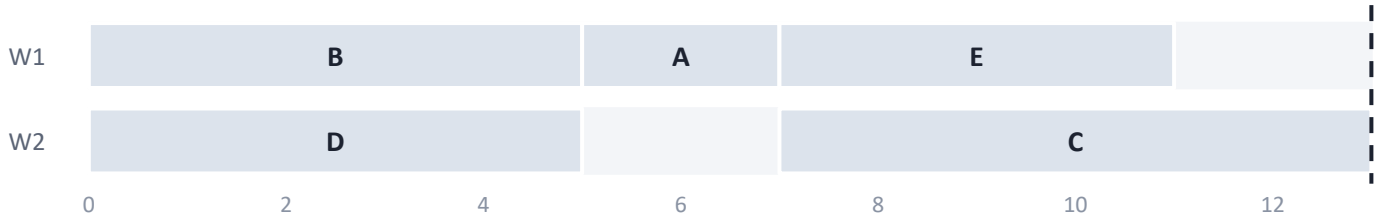
Each box is a task; its number is its duration. An arrow X → Y means Y cannot start until X has finished.

TWO IDENTICAL WORKERS, ONE TIMELINE

A good schedule **Cmax = 11**



A plausible but worse schedule **Cmax = 13**



- **Online decision:** whenever a worker becomes free, pick one of the ready tasks.
- **Ready:** all of a task's prerequisites are finished.
- **Goal:** minimise the time the last task finishes (the makespan, Cmax).

Mathematical formulation

The classical problem P | prec | Cmax, which the scheduler solves online with deterministic durations.

Tasks and DAG	G = (V, E) is a directed acyclic graph; (u, v) ∈ E means u must finish before v starts.
Durations	Each task v has an integer duration d(v) > 0, known in advance.
Workers	m identical workers (P); each runs one task at a time.
Precedence	$S(v) \geq S(u) + d(u)$ *for every edge* $(u, v) \in E$.
Non-preemptive	*Once started, v runs without interruption:* $C(v) = S(v) + d(v)$.
Objective	*minimise* $Cmax = \max$ *over* $v \in V$ *of* $C(v)$.

Three-field notation



identical parallel machines

precedence constraints (a DAG)

makespan objective

A simple lower bound used throughout

$$LB = \max(CP , \lceil \Sigma d(v) / m \rceil)$$

CP is the longest dependency chain (no schedule can beat it); Σd/m is the total work spread perfectly over m workers. Every schedule satisfies Cmax ≥ LB, but the optimum can sit strictly above LB.

Why is the problem NP-hard? A reduction from PARTITION

Even with no dependencies and only two workers, deciding whether a schedule meets a deadline is as hard as PARTITION.

1 PARTITION

Given positive integers $a_1 \dots a_n$ with sum S, can they be split into two sets of equal sum S/2? (NP-complete; Karp 1972)

2 Construction

For each a_i create a task of duration a_i. No precedence edges. m = 2 identical workers. Deadline T = S/2.

3 ⇒

If a partition exists, run one set on each worker: both finish at S/2, so Cmax ≤ T.

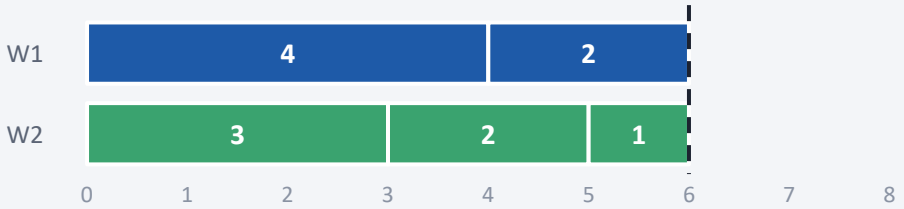
4 ⇐

If Cmax ≤ S/2, the two workers' loads sum to S and each is ≤ S/2, so each is exactly S/2: their task sets form a partition.

5 Polynomial

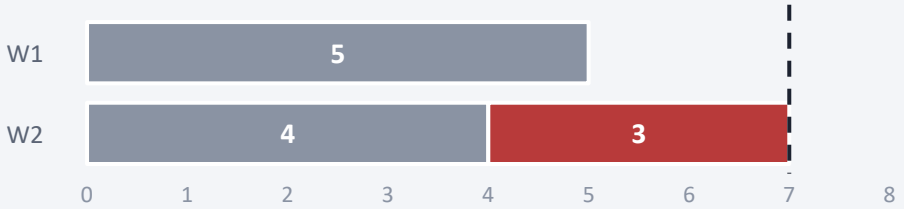
The construction copies n numbers: O(n) time. Hence the decision version of P2 || Cmax (and so of P | prec | Cmax) is NP-hard.

YES-instance {4, 3, 2, 2, 1} · S = 12, T = 6



{4, 2} | {3, 2, 1}: both workers finish at 6 = T. Exact solver: optimum 6.

NO-instance {5, 4, 3} · S = 12, T = 6



No subset sums to 6, so every schedule has Cmax ≥ 7 > T. Exact solver: optimum 7.

Each bar is one number / task, labelled by its duration.

What the reduction does and does not say

Precise claims, because NP-hardness is often over-interpreted.

Decision version

"Is there a schedule with Cmax ≤ T?" is NP-complete: a schedule is a short certificate checked in polynomial time.

Optimisation version

Minimising Cmax is NP-hard: an optimal algorithm would also answer the decision question.

Weak, not strong

PARTITION is weakly NP-complete (pseudo-polynomial algorithms exist). This reduction proves weak NP-hardness only; strong NP-hardness needs a different reduction (e.g. from 3-PARTITION).

What NP-hardness does not mean

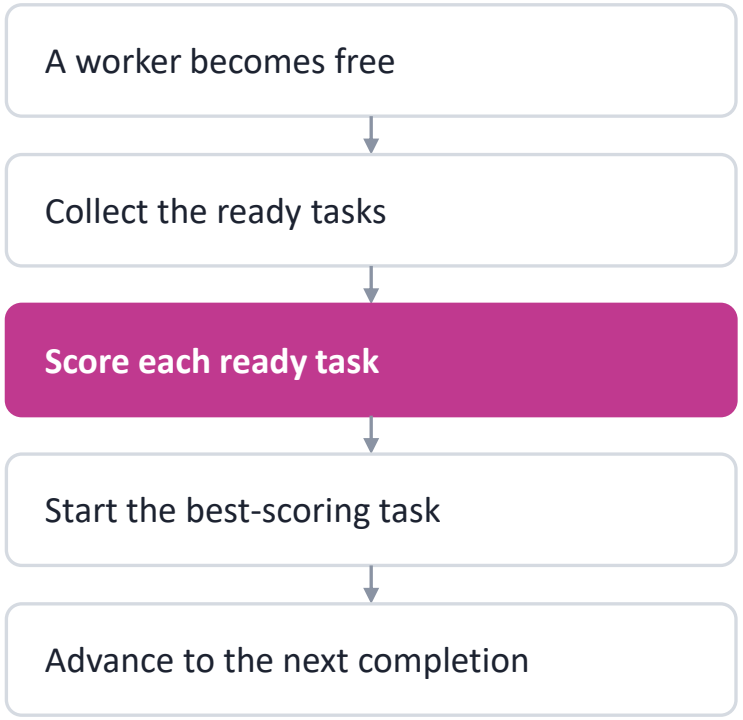
- **Not "every instance is hard".** In our exact experiments, Critical Path First is already optimal on 93.6% of 800 tiny instances (2–10 tasks).
- **Not "optimum cannot be computed".** Our branch-and-bound solver proves optima for 2–18-task graphs; a single 19–22-task instance took up to 2.5 h (Phase 3).
- **It does mean:** no known algorithm guarantees an optimum in polynomial time for every instance, so practical schedulers rely on fast, well-tested priority rules.

Why we need a heuristic

Our requirements point to a priority-based list scheduler.

Online	Decide at every worker-free event; durations and the DAG are known but decisions are made as time advances.
Fast	Thousands of tasks; decisions in milliseconds, not hours of exhaustive search.
Robust	Graphs of very different shapes, sizes, duration mixes and worker counts.
Explainable	Each decision should come from interpretable signals about the DAG and the current state.

List scheduling, the template we use



A list scheduler never idles a worker while a task is ready. On small graphs the best such schedule is within 0.02–0.03% of the true optimum (exact experiments, Phases 1 and 3).

The whole research question is step 3: which score?

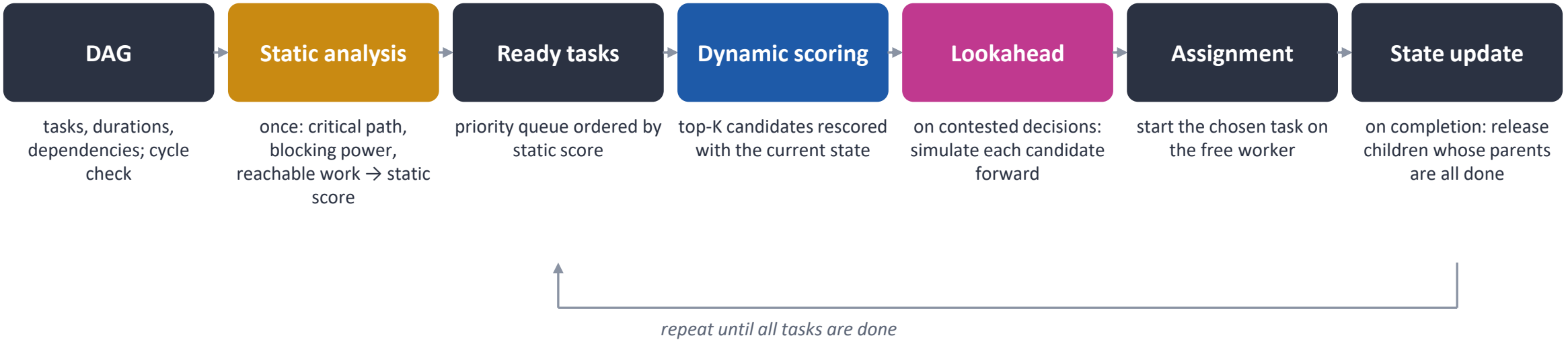
II

Our scheduling algorithm

Static graph analysis, dynamic state-aware scoring, and how they were combined in the original design.

Overall architecture

What happens every time a worker becomes available.



Original design (Original scheduler)

Top K = 3 tasks by static score → blended score ½·static + ½·dynamic → an idle-window lookahead compared the best two when their scores differed by more than 0.1.

Final design (QUALITY_OPTIMIZED)

Top K = 5 by static score → learned blend (λ ≈ 0.999) → rollout lookahead on every contested decision (more ready tasks than free workers), depth 8, 5 candidates.

Static component 1 · Critical path

The length of the longest dependency chain that starts at the task.

DEFINITION (AS IMPLEMENTED)

$$cp(v) = d(v) + \max\{\, cp(c) : c \in children(v) \,\}$$
$$cp(v) = d(v) \text{ for tasks with no children}$$

TimeConstraints.criticalPaths(); normalised by the overall critical-path length.

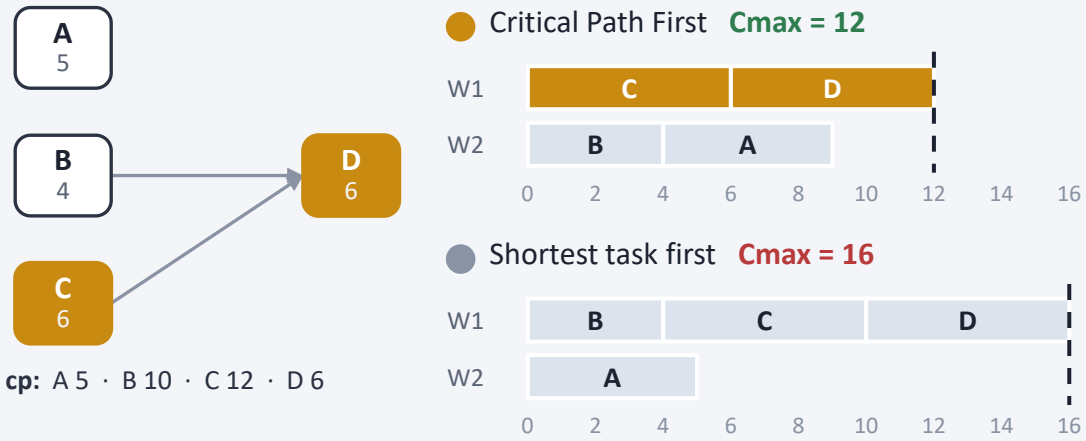
Measures

The remaining “bottom-level” time: how long the work hanging below v takes even with unlimited workers.

Why we expected it to help

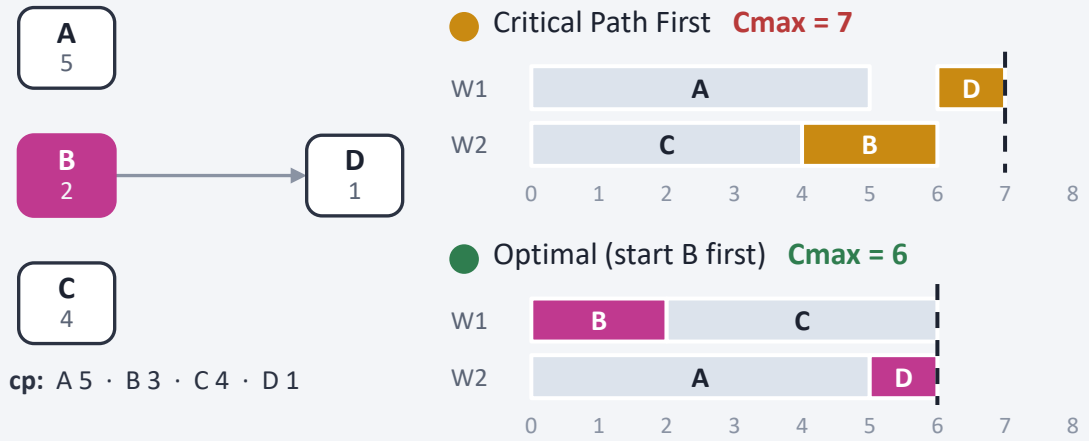
Delaying a task on a long chain delays everything after it, so the whole project finishes later. Starting long chains early protects the critical path.

WHERE IT HELPS 2 workers · C → D is the long chain



CP starts C (cp 12) at once; shortest-first delays it and D waits until 10. 12 is optimal.

WHERE IT FAILS 2 workers · the short task B unlocks D



Misses when workers free up: B (cp 3) loses to C (cp 4), so D is released late and a worker idles at t = 5. Optimum 6.

Static component 2 · Blocking power

How much of the remaining graph waits directly on this task.

DEFINITION (AS IMPLEMENTED)

$$bp(v) = \sum_{c \in children(v)} 1 / |parents(c)|$$

Measures

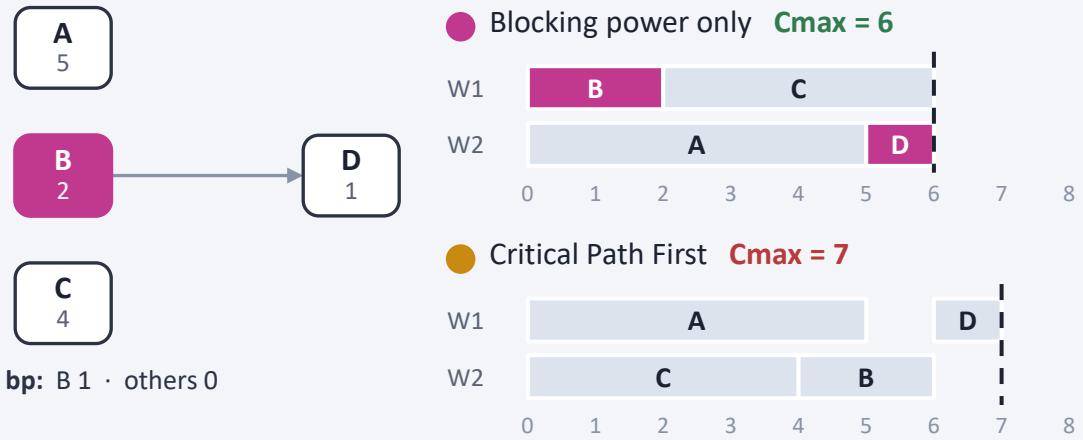
Each child contributes its share of “unblocking”: a sole-parent child counts 1, a child with k parents counts 1/k.

Why we expected it to help

Finishing tasks that unblock others keeps the ready queue full, so workers are less likely to sit idle.

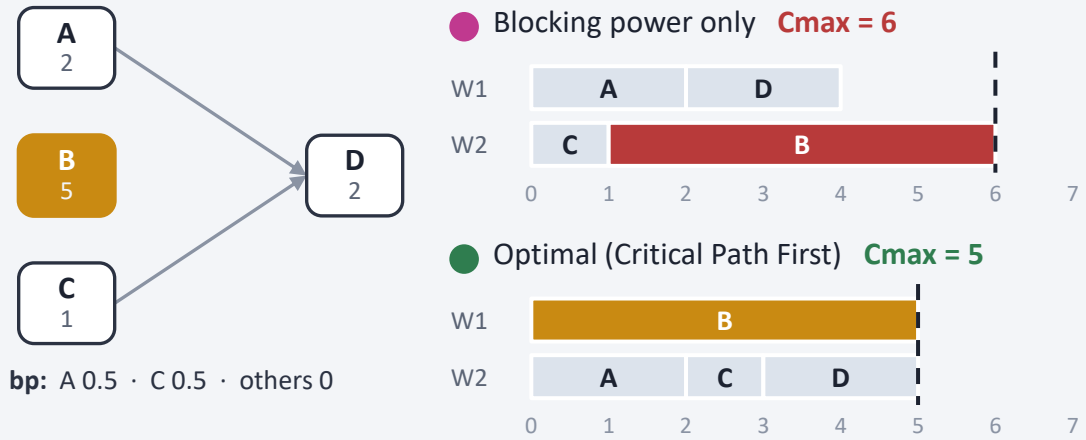
TimeConstraints.blockPower(); normalised by the maximum bp in the DAG. Ties are broken by task id.

WHERE IT HELPS same instance where CP failed



bp(B) = 1, everything else 0: B goes first, D is released at t = 2 and the schedule reaches the optimum 6.

WHERE IT FAILS 2 workers · B is long but unblocks nothing



Misses durations: A and C (bp ½ each) beat B (bp 0), so the 5-unit task B starts late. Optimum 5.

Static component 3 · Reachable workload

The total work that depends, directly or indirectly, on this task.

DEFINITION (AS IMPLEMENTED)

$$\text{reach}(v) = \sum_{u \in \{v\} \cup \text{descendants}(v)} d(u)$$

Measures

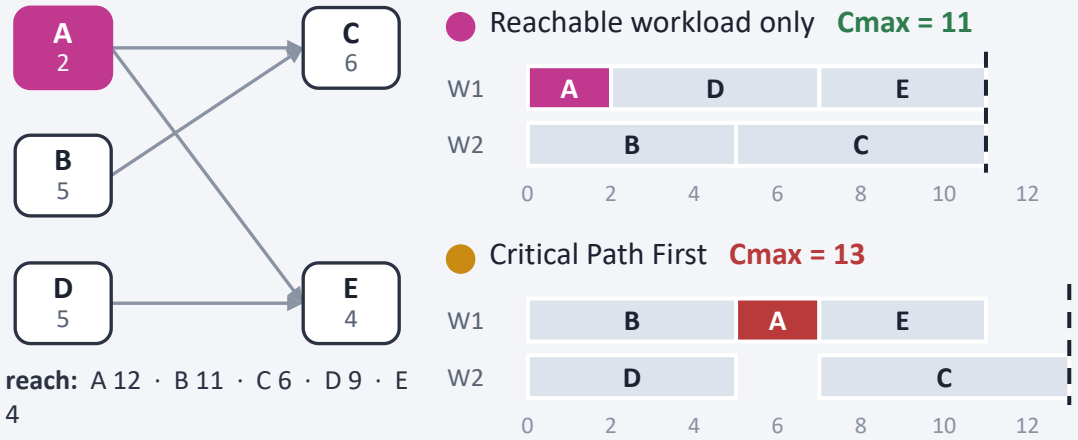
The volume of work downstream of v, including v itself. Overlapping descendants are counted once.

Why we expected it to help

A short task that feeds a large amount of downstream work should run early, even if no single chain below it is long.

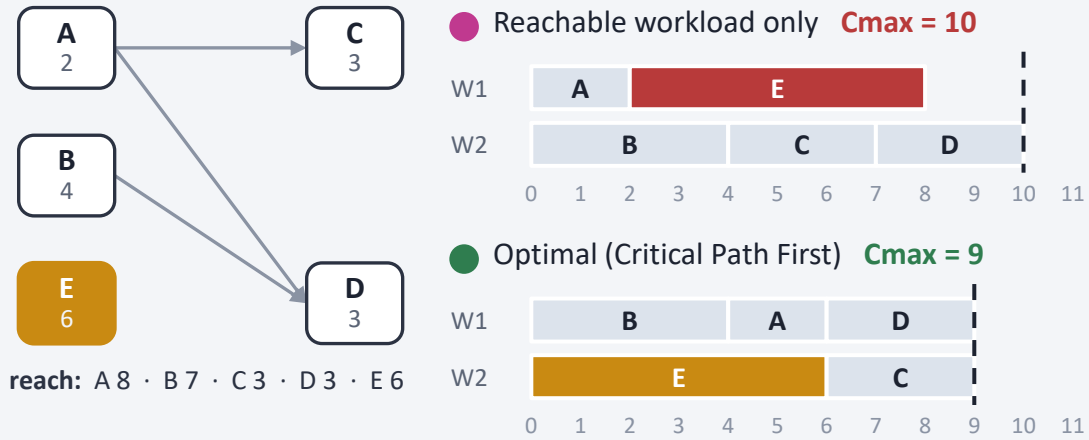
TimeConstraints.reachableDuration(); normalised by total work (original) or by max reach (optimised presets).

WHERE IT HELPS 2 workers · A feeds both C and E



reach(A) = 12 is the largest, although cp(A) = 8 is the smallest. Starting A first gives the optimum 11; CP gets 13.

WHERE IT FAILS 2 workers · E is long and independent



Misses chain length and timing: A (8) and B (7) outrank E (6), so the 6-unit E starts late. Optimum 9.

Combining the static components

One normalised score per task, computed once before scheduling starts.

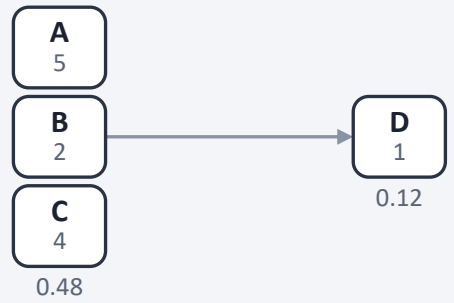
$$static(v) = \mathbf{0.50} \cdot cp(v) / cp_max \; + \; \mathbf{0.25} \cdot bp(v) / bp_max \; + \; \mathbf{0.25} \cdot reach(v) / W$$

cp_max = overall critical-path length · bp_max = largest blocking power · W = total work Σ d(v). Optimised presets divide reach by max reach instead of W (Phase 1, Change 11).

Why combine instead of choosing one?

- Each component sees one kind of information and has its own blind spot (previous three slides).
- Normalising to [0, 1] makes them comparable across graphs of any size.
- Weights express a prior: chain length matters most, unblocking and downstream volume second.
- **The weights 0.5 / 0.25 / 0.25 were a design choice, not derived or tuned.** Testing that choice is Part IV.

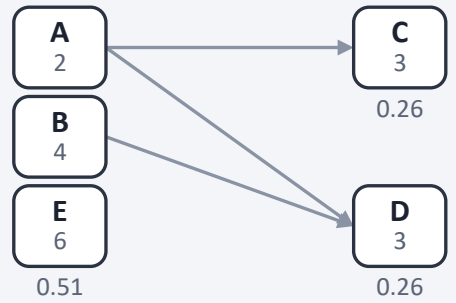
Fixes the CP failure



Combined static score: Cmax 6 (optimal; CP-only gives 7)

numbers under tasks = static score

But inherits the reach failure



Combined static score: Cmax 10 (optimum 9)

numbers under tasks = static score

Dynamic components: scoring the current state

Five signals recomputed at every decision for the top-K ready tasks. They depend on what is running and what is ready now.

component	weight	intent	formula (DynamicScoreLogic.makeDynamicScore)
openToBlock	**0.20**	share of v's children that v alone still blocks	$open(v) \,/\, \lvert children(v) \rvert$
openChildren	**0.10**	children released by v, relative to the worker count	$min(1, open(v) \,/\, m)$
childImportance	**0.15**	average static score of the children v would release	$\Sigma\, static(c) \,/\, (open(v) \cdot static_max)$
durationToReady	**0.30**	long tasks when many are ready, short tasks when few are	$\alpha \cdot \hat{d}(v) + (1-\alpha)(1-\hat{d}(v))$
childrenToReadyQueue	**0.25**	children released, but only when the ready queue is short	$(1-\alpha) \cdot open(v) \,/\, maxChildren$

Shared notation. open(v) = number of children c whose only unfinished parent is v (|parentsRemaining(c)| = 1). m = total workers. α = min(1, |ready| / m) is the workload pressure. $\hat{d}(v) = d(v) / d_max$. maxChildren = the largest out-degree in the DAG. dynamic(v) = 0.2·openToBlock + 0.1·openChildren + 0.15·childImportance + 0.3·durationToReady + 0.25·childrenToReadyQueue.

Dynamic 1 · Open-to-block ratio

What fraction of my children am I the last thing blocking?

DEFINITION (AS IMPLEMENTED)

openToBlock(v) = open(v) / |children(v)|
(0 if v has no children)

Measures

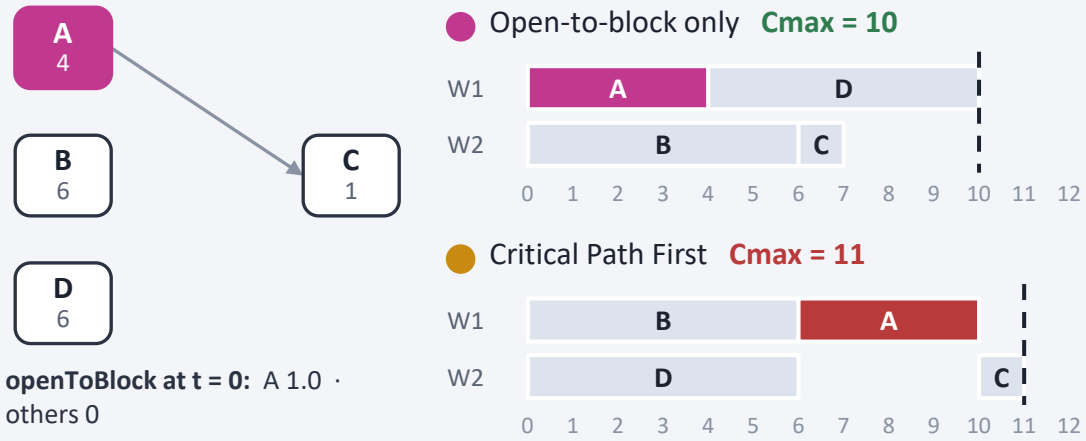
Whether finishing v would actually release work now, rather than just reduce a child's parent count.

Why we expected it to help

A task that is the final blocker of its children turns into new ready work the moment it ends, which helps avoid idle workers.

Illustration policy: rank all ready tasks by this component alone (λ = 0, K = all, no lookahead, batched completions); ties fall back to static order, as in the implementation.

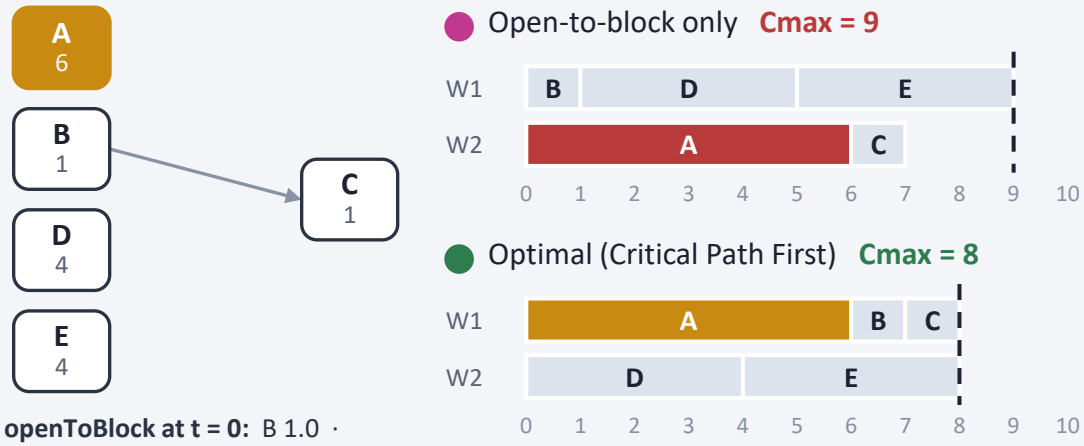
WHERE IT HELPS 2 workers · A is the last blocker of C



openToBlock at t = 0: A 1.0 · others 0

A scores 1.0, B and D score 0, so A starts at t = 0 and C is ready early. Optimum 10; CP (B, D first) gets 11.

WHERE IT FAILS 2 workers · B unlocks only a 1-unit task



openToBlock at t = 0: B 1.0 · others 0

Misses how much the released work matters: B (1.0) takes a worker at t = 0, so D and E (4 units each) queue behind it on one worker. Optimum 8.

Dynamic 2 · Released children vs. workers

Will finishing me release enough work to keep the other workers busy?

DEFINITION (AS IMPLEMENTED)

$$\text{openChildren}(v) = \min(1, \text{open}(v) / m)$$

Measures

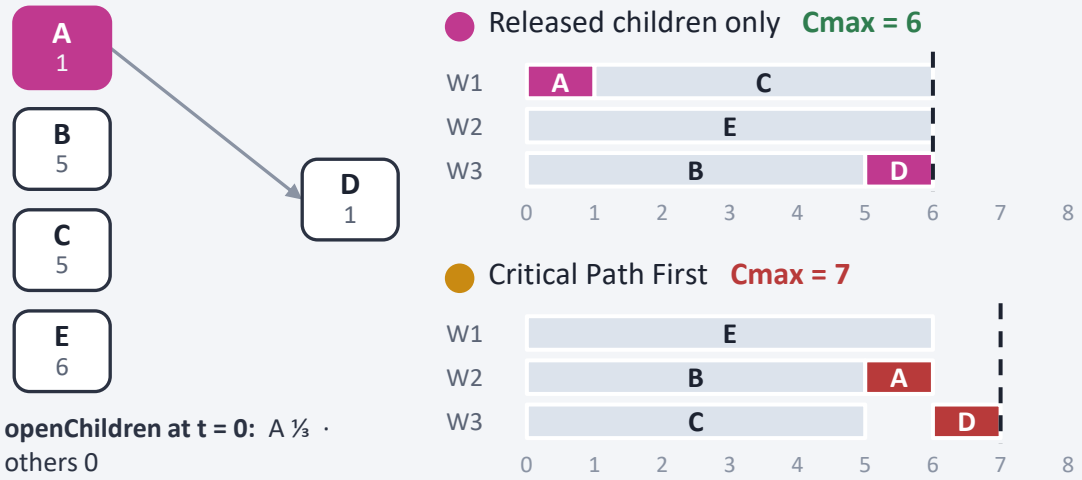
The number of children v would release, as a share of the worker count m and capped at 1.

Why we expected it to help

If m workers are about to free up, a task that releases several children can feed them, which reduces starvation.

Illustration policy: rank all ready tasks by this component alone (λ = 0, K = all, no lookahead, batched completions); ties fall back to static order, as in the implementation.

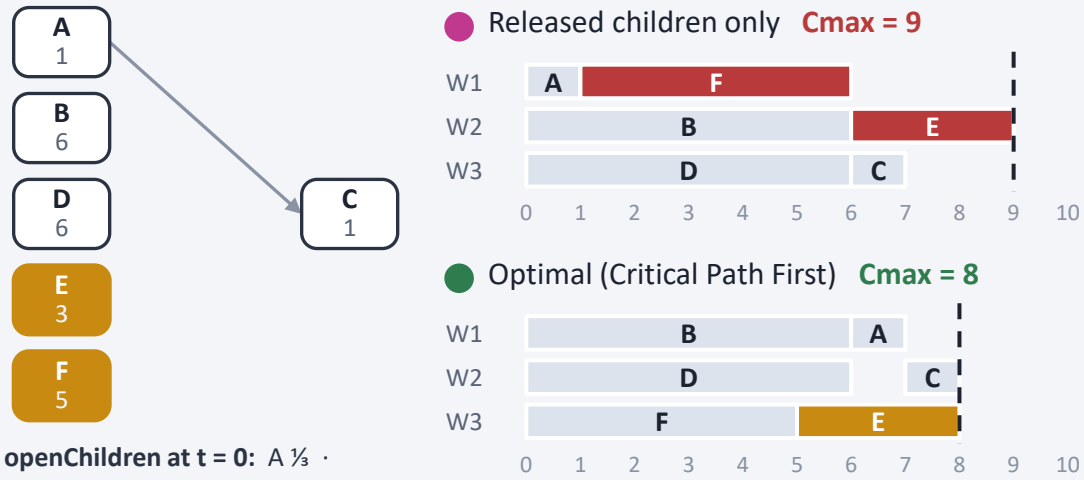
WHERE IT HELPS 3 workers · A releases D



openChildren at t = 0: A ⅓ · others 0

A (⅓) starts at t = 0 with E and B, so D is ready at t = 1. Optimum 6; CP starts E, B, C, delays A to t = 5, and gets 7.

WHERE IT FAILS 3 workers · A releases only a 1-unit task



openChildren at t = 0: A ⅓ · others 0

Misses durations: A (⅓) takes a worker at t = 0 ahead of F (5) and E (3), pushing E to t = 6. Cmax 9 vs optimum 8.

Dynamic 3 · Child importance

Are the children I release themselves important?

DEFINITION (AS IMPLEMENTED)

childImportance(v) = mean of static(c) over open children c, ÷ static_max (0 if none)

Measures

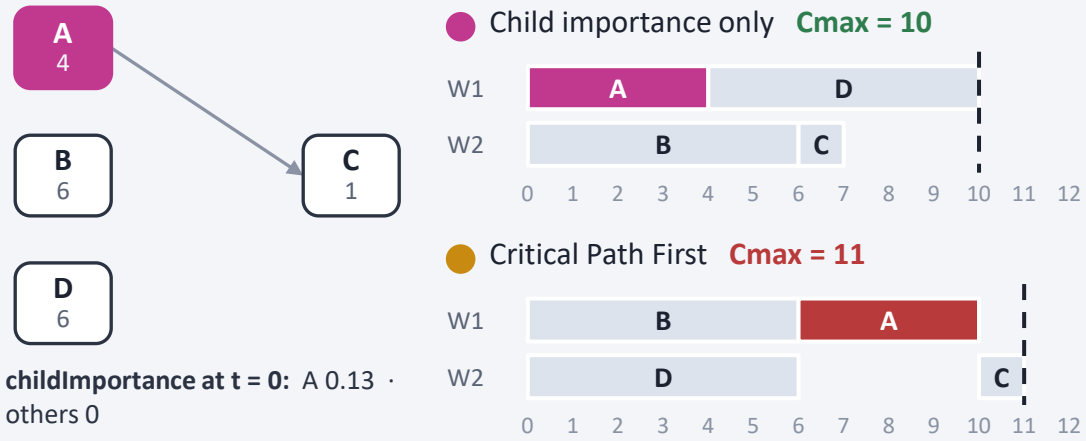
The average static score of the children v would release, normalised by the highest static score in the DAG.

Why we expected it to help

Releasing a child that sits on the critical path should matter more than releasing a leaf.

Illustration policy: rank all ready tasks by this component alone (λ = 0, K = all, no lookahead, batched completions); ties fall back to static order, as in the implementation.

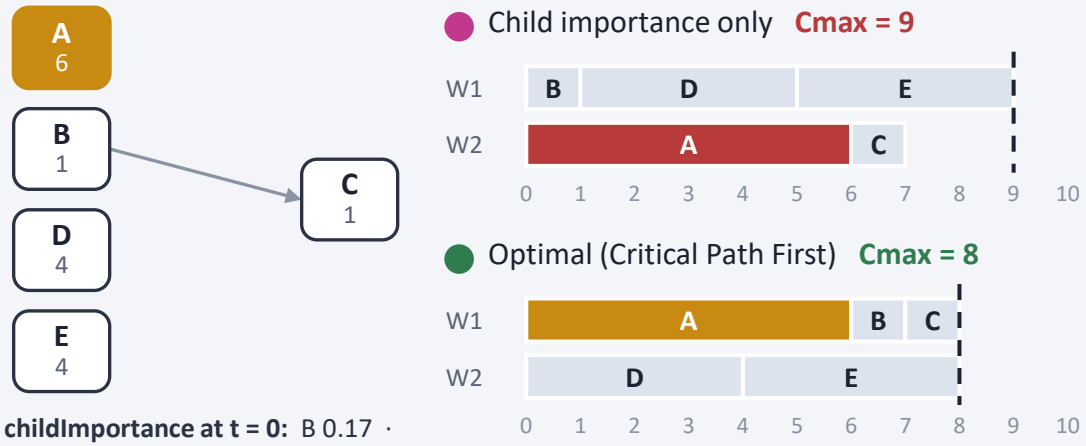
WHERE IT HELPS 2 workers · same instance as Dynamic 1



childImportance at t = 0: A 0.13 · others 0

Even a modest child score (0.13) beats the 0 of tasks that release nothing. Same schedule as open-to-block. Optimum 10.

WHERE IT FAILS 2 workers · same instance as Dynamic 1



childImportance at t = 0: B 0.17 · others 0

Misses the releasing task's own urgency: any open child (0.17) beats 0, so the 1-unit B takes a worker ahead of D and E. Cmax 9 vs optimum 8.

Dynamic 4 · Duration-aware scoring

Prefer long tasks under heavy load, short tasks under light load.

DEFINITION (AS IMPLEMENTED)

$$\text{durationToReady}(v) = \alpha \cdot \hat{d}(v) + (1-\alpha)\cdot(1-\hat{d}(v))$$
$$\alpha = \min(1, |\text{ready}| / m), \quad \hat{d} = d / d_{max}$$

Measures

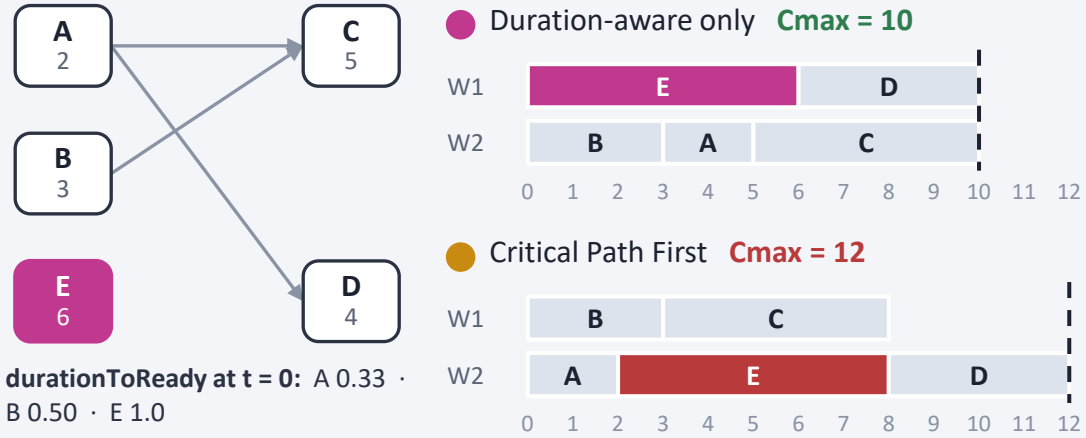
When at least m tasks are ready (α = 1) it ranks by duration, like LPT. With few ready tasks it ranks short tasks first.

Why we expected it to help

Under load, starting long work early avoids a long tail. When work is scarce, short tasks finish quickly and may release more.

Illustration policy: rank all ready tasks by this component alone (λ = 0, K = all, no lookahead, batched completions); ties fall back to static order, as in the implementation.

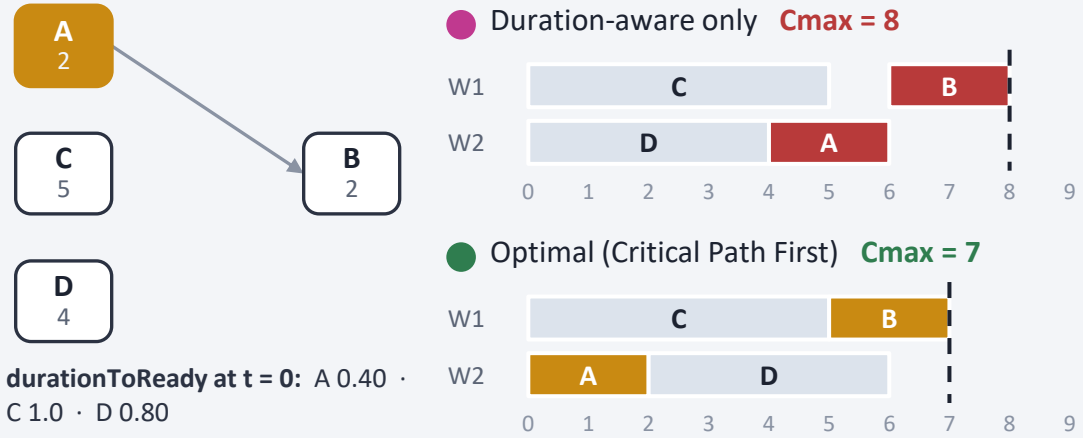
WHERE IT HELPS 2 workers · E is long and independent



durationToReady at t = 0: A 0.33 · B 0.50 · E 1.0

Three ready tasks for 2 workers, so α = 1: E (6 units) starts at once. Optimum 10; CP starts E at t = 2 and gets 12.

WHERE IT FAILS 2 workers · A heads a chain A → B



durationToReady at t = 0: A 0.40 · C 1.0 · D 0.80

Misses dependencies: as pure LPT it starts C and D, delaying the chain head A. Cmax 8 vs optimum 7.

Dynamic 5 · Children to ready queue

Release new work when the ready queue is running dry.

DEFINITION (AS IMPLEMENTED)

childrenToReadyQueue(v) = (1−α) · open(v) / maxChildren

Measures

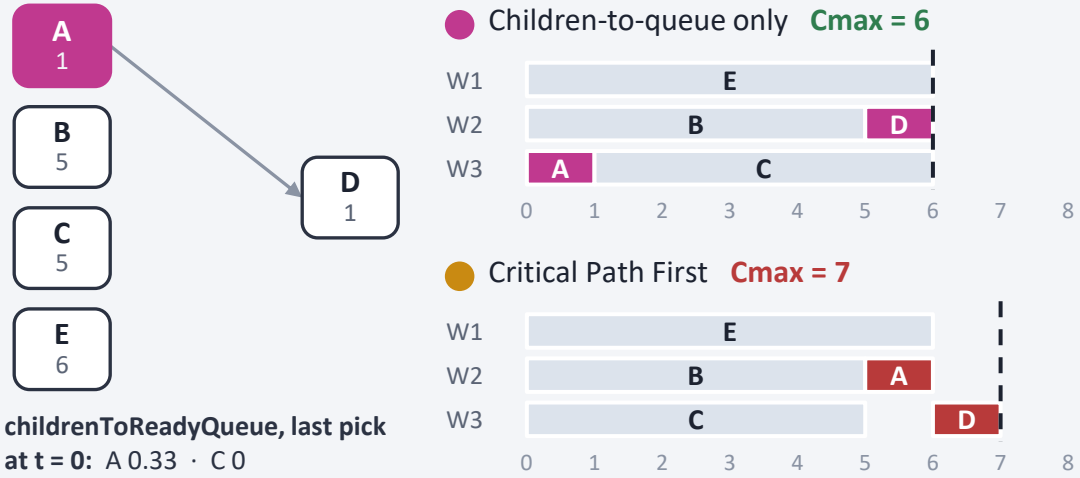
Released children scaled by maxChildren (largest out-degree). Exactly 0 whenever |ready| ≥ m, i.e. in most heavily contested decisions.

Why we expected it to help

When fewer tasks are ready than workers exist, the priority is to refill the queue.

Illustration policy: rank all ready tasks by this component alone (λ = 0, K = all, no lookahead, batched completions); ties fall back to static order, as in the implementation.

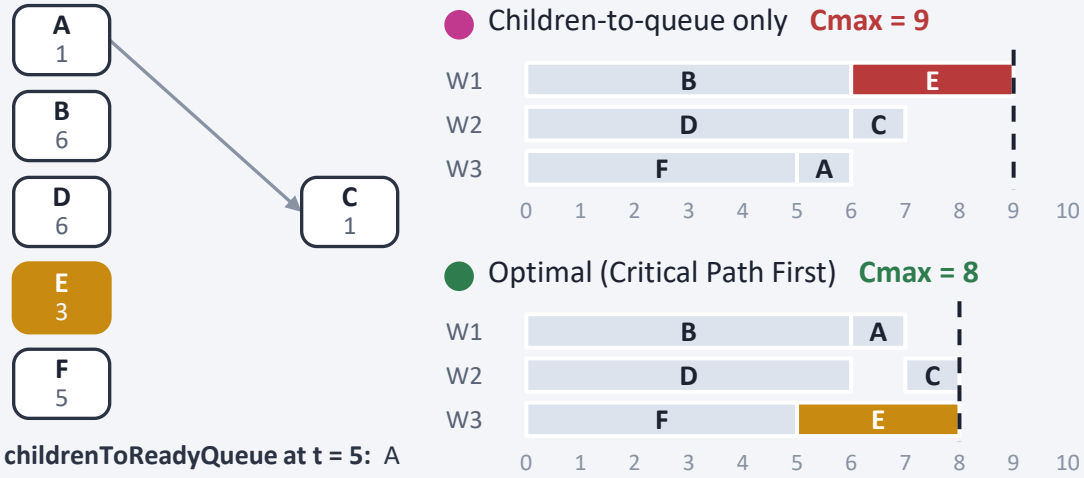
WHERE IT HELPS 3 workers · 2 ready at the third pick → α = ⅔



childrenToReadyQueue, last pick at t = 0: A 0.33 · C 0

With 2 tasks ready for the last free worker, A scores ⅓ and C scores 0, so A goes first. Optimum 6; CP gets 7.

WHERE IT FAILS 3 workers · decision at t = 5, 2 ready

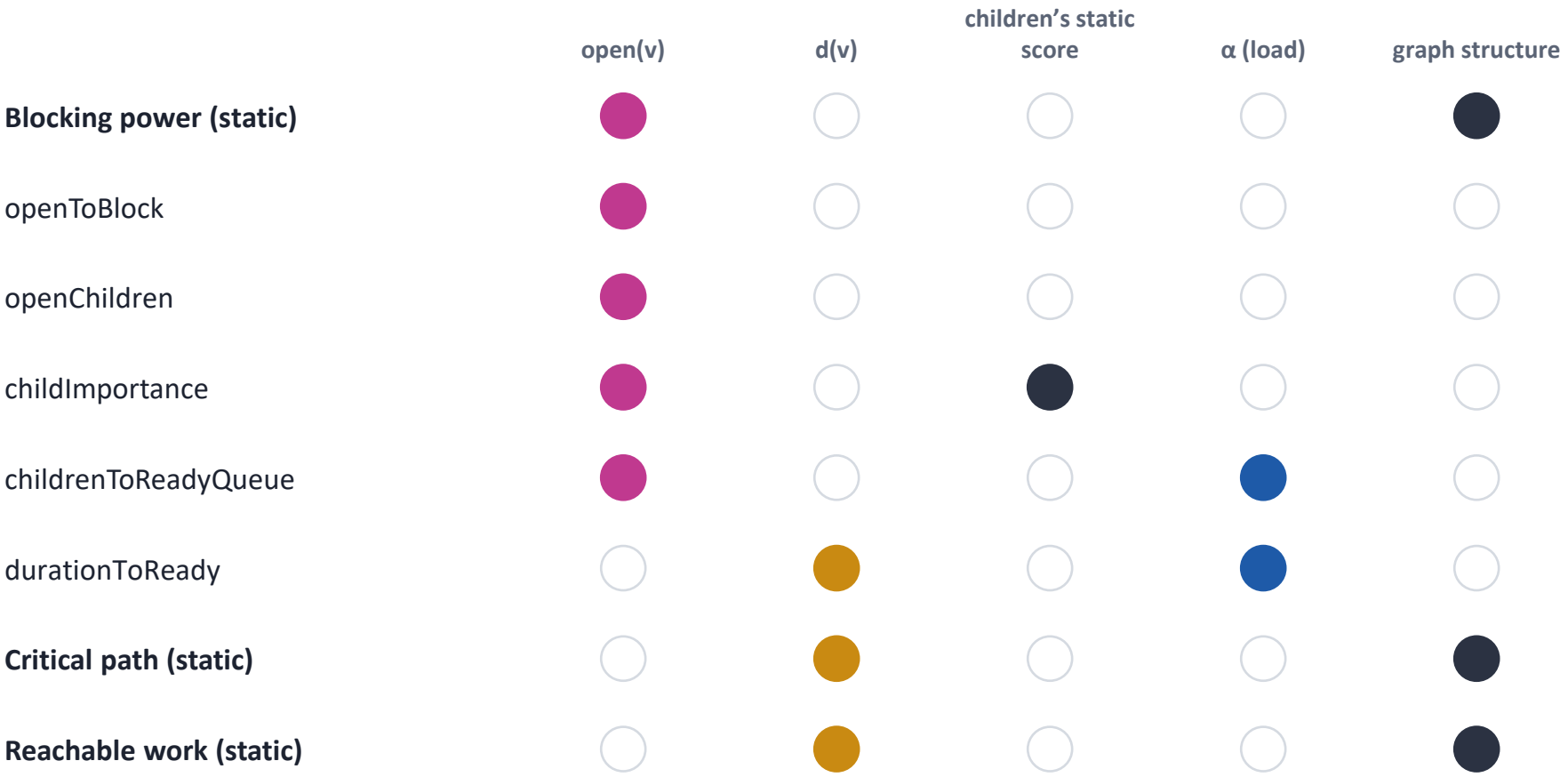


childrenToReadyQueue at t = 5: A 0.33 · E 0

Misses durations: A (⅓) beats E (0) although E is 3 units long. Cmax 9 vs optimum 8.

The components overlap

Most dynamic signals are different scalings of one quantity, open(v): the work v would release now.



Consequences

- Four dynamic signals rise and fall together with open(v).
- On our verified examples the three unlocking components produced identical schedules.
- Correlated features make individual weights hard to identify. Optimisation later finds a flat plateau (Part IV).

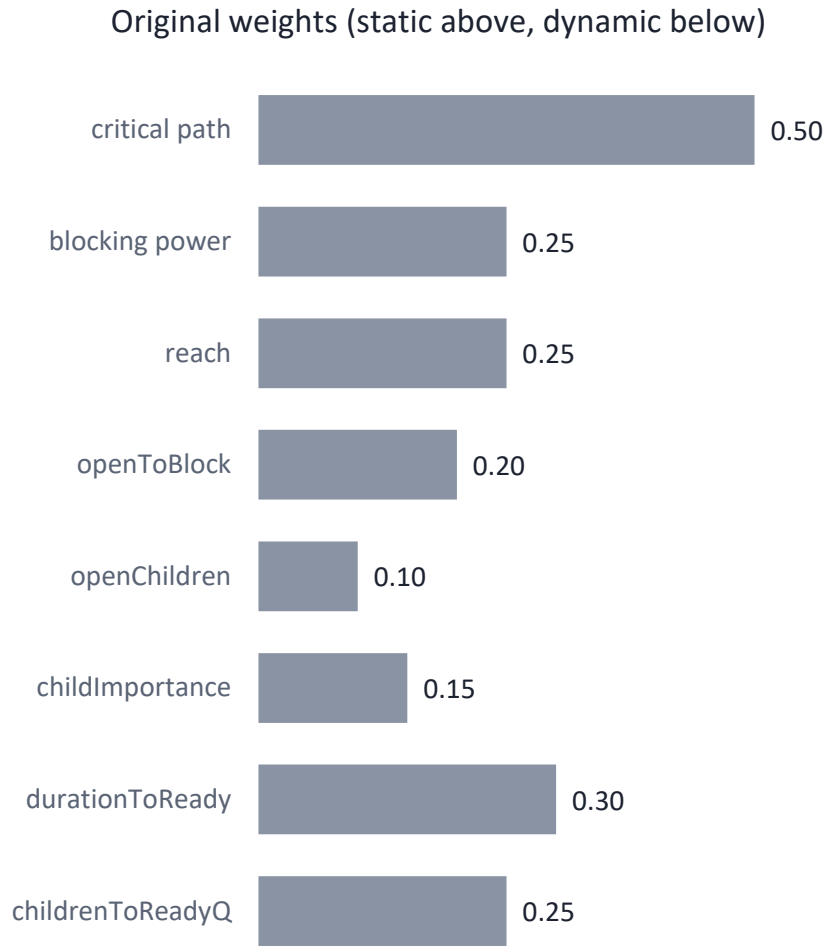
Combining static and dynamic scores

The original blend, and how many candidates it considered.

$$score(v) = \lambda \cdot static(v) + (1 - \lambda) \cdot dynamic(v)$$

Original λ = 0.5, i.e. (static + dynamic) / 2

- **Candidate pool.** Only the K = 3 best ready tasks by static score are rescored dynamically (K is configurable in the research code).
- **Blending factor λ** sets how much the current state may override the graph structure: λ = 1 is pure static priority, λ = 0 is pure dynamic.
- **Original lookahead.** If the top two blended scores differed by more than 0.1 under the load gate, an idle-window estimate was added. This is replaced by the rollout in the final design (Part III).
- **All initial weights were heuristic choices:** static 0.5 / 0.25 / 0.25, dynamic 0.2 / 0.1 / 0.15 / 0.3 / 0.25, λ = 0.5. None were proven or tuned.



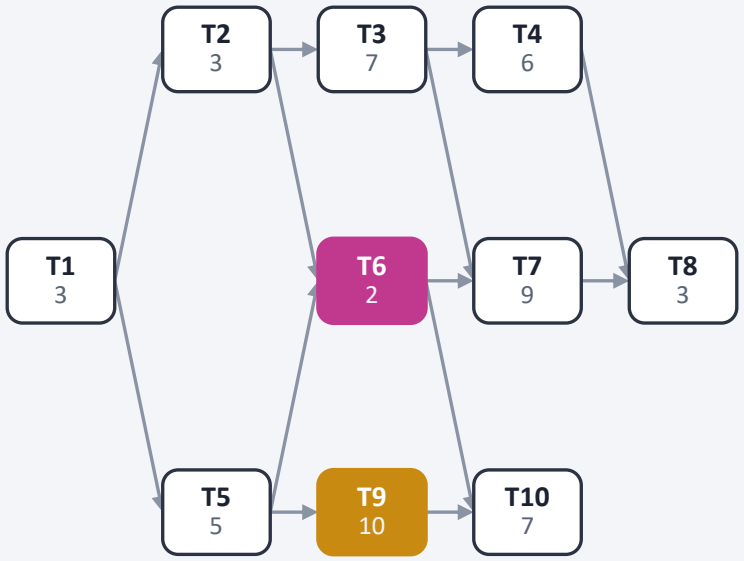
III

Final lookahead

Local scores cannot see when workers become free. A bounded rollout can.

Why local scoring is not enough: worker starvation

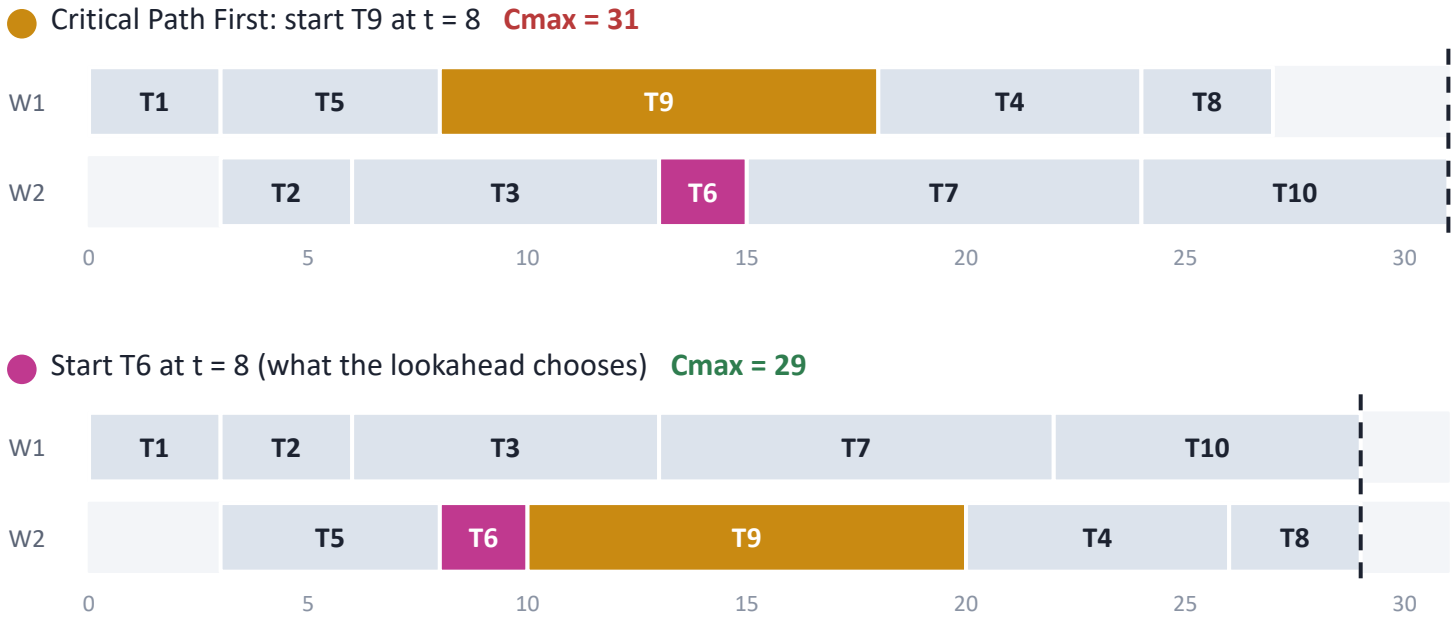
A real benchmark instance (Phase 3, exact set): 10 tasks, 2 workers. Critical Path First: 31. Optimum: 29.



At t = 8 one worker is free and two tasks are ready:

T9 cp 17 · duration 10 · releases nothing alone
T6 cp 14 · duration 2 · co-parent of T7 and T10

Critical-path order says T9. But T7 needs T6, and T7 is what the other worker should run when T3 ends at t = 13.



What went wrong. With T9 first, T6 waits until t = 13, T7 starts at 15 and T10 at 24, and one worker sits idle from 27 to 31. Starting the short T6 first costs T9 two units of slack but lets T7 start at 13. The difference comes from the timing of future completions, which no static or one-step score sees.

Our final lookahead: bounded rollout

Used in PHASE1_ROLLOUT and QUALITY_OPTIMIZED; validated in Phases 1–3.

1	**Activate**	Only on contested decisions: more ready tasks than workers able to start one now (idle workers, the deciding one, and those finishing at this instant).
2	**Select candidates**	Take the K = 5 best ready tasks by static score, order them by blended score, and keep the first L = 5. The static best is always included.
3	**Simulate**	For each candidate: start it now, then continue with deterministic static-priority list scheduling (ties by id). Simultaneous completions are applied together; idle workers take ready tasks at once.
4	**Bound the horizon**	Stop after 8 completion events (depth 8). Running tasks keep their true remaining times.
5	**Evaluate**	If the simulation finished, use its exact makespan. Otherwise use a lower bound on the final makespan of the reached state (right).
6	**Choose**	Start the candidate with the smallest estimate. Ties keep the blended-score order. The real ready queue and workers are never modified.

STATE EVALUATION AFTER d EVENTS

$$\hat{C} = \max(\ \max_{j \text{ running}} \mathrm{end}(j) + \mathrm{cp}(j) - d(j),\ \max_{j \text{ ready}} t + \mathrm{cp}(j),\ t + \lceil \text{remaining work} / m \rceil\ )$$

A valid lower bound on the makespan reachable from the simulated state: chain bound plus work bound.

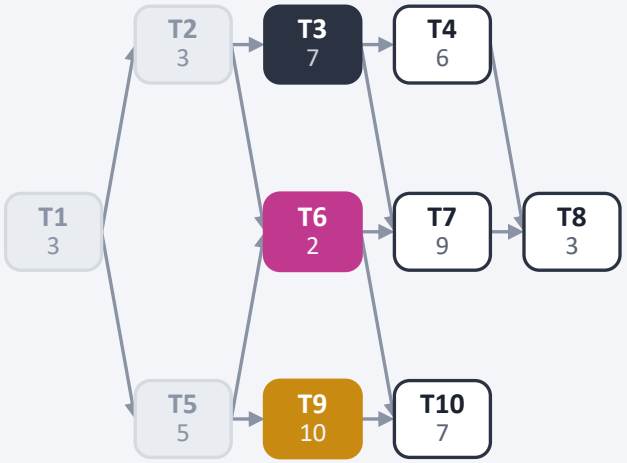
Why this replaced the original lookahead

Every candidate is compared on one common quantity, the estimated final makespan, rather than candidate-specific idle windows.

Lookahead in action

QUALITY_OPTIMIZED on the instance from the previous slide, t = 8. Numbers are the real rollout outputs (instrumented run).

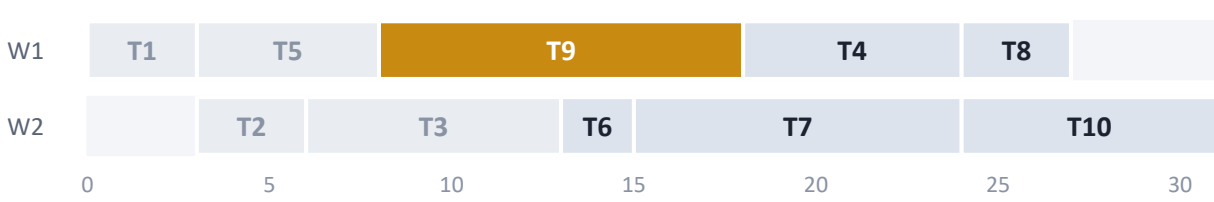
CURRENT STATE · t = 8



- **Done:** T1, T2, T5
- **Running:** T3 on W1 until t = 13
- **Free:** W2
- **Ready:** T9, T6 → contested (2 ready, 1 free)
- **Blended score:** T9 0.680 > T6 0.560

SIMULATED FUTURES (8-event rollout, continuation by static priority)

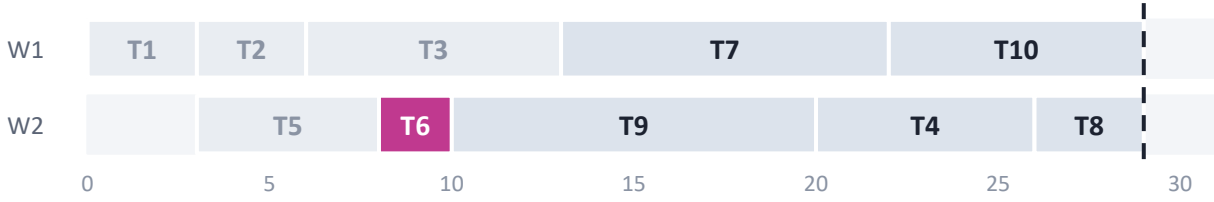
Candidate T9 **Cmax = 31**



31

exact makespan (finishes within 8 events)

Candidate T6 **Cmax = 29**



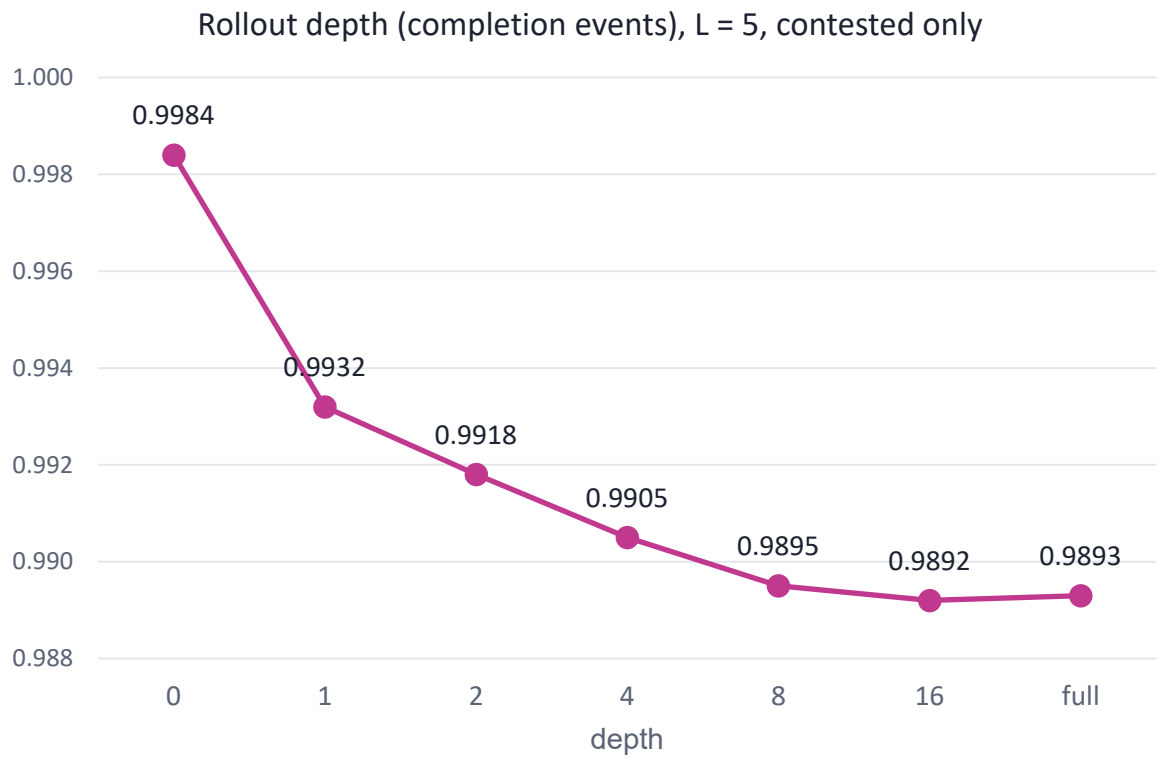
29

exact makespan (finishes within 8 events)

Decision: start T6 (29 < 31), overriding the blended-score ranking. Two later contested decisions are also resolved by rollout: t = 13 T7 (29) vs T4 (31), and t = 20 T4 (29) vs T10 (31). Final schedule: 29, the optimum proven by the exact solver; Critical Path First and the Original scheduler both give 31.

Depth, candidates and cost

Deeper search helps up to about 8 events, then quality flattens while cost keeps growing (Phase 1, MAIN set, 3,600 instances).



Lower is better. Geometric-mean (GM) ratio of makespan to the Original scheduler on the same instances.

single thread, ms per run	S	M	L	XL
Original	0.11	1.1	1.4	3.5
depth 8, L = 5	**0.08**	**1.5**	**20**	**159**
depth 16, L = 5	0.09	2.0	28	229
full depth, L = 5	0.10	4.1	88	801

S 10–30, M 31–150, L 151–600, XL ≈ 450–1,500 tasks (Phase 1 §7–8).

Validated

Depth 8, L = 5, contested-only. On XL graphs depth 8 (0.9945) beat full depth (0.9953). L = 5 helps only at depth ≥ 8. Activation is decisive: under the original gates the rollout reached only 0.9956.

Not validated

Adaptive depth or a graph-level gate inside the rollout. Phase 3 tested such gates only in a study simulator, or as preset switching by graph size; neither is implemented in production.

IV

Optimizing the weights

How should weight be distributed among the components to minimise makespan across diverse problems?

The research question

Nine continuous knobs and three discrete ones; no theory says how to set them.

Static weights	w_cp · w_bp · w_reach	*simplex: ≥ 0, sum 1*
Dynamic weights	openToBlock · openChildren · childImp · dur→ready · children→queue	*simplex: ≥ 0, sum 1*
Blend	λ	*[0, 1]*
Structure	K pool · L candidates · depth	*discrete grid*

Why manual tuning is hard

- Effects interact: a dynamic weight matters only if λ lets it override the static order.
- The objective is piecewise constant: small weight changes often change no decision at all.
- Each component helps on some graphs and hurts on others (Part II), so judging by examples misleads.

Our approach: large-scale empirical optimisation

Generate thousands of diverse DAGs. Search the weight space on training graphs, select on separate validation graphs, and report once on untouched test graphs, with paired statistics against the Original scheduler.

Experimental methodology

Phase 2 datasets. All seeded, deterministic, disjoint across roles and from the Phase 1 seeds.

INSTANCE GENERATOR

- **8 DAG families:** Erdős–Rényi, layered, fork-join, tree, chains, stencil, FFT, Gaussian elimination
- **5 duration distributions:** uniform 1–10, uniform 1–100, log-normal, unit, bimodal
- **Sizes:** S 10–30, M 31–150, L 151–600, XL 601–1,500 tasks
- **Workers:** 2–64, set from average parallelism × pressure ∈ {0.5, 1, 1.5, 3}

Train	seed 10001	7,200	6 families × S/M/L, no BIMODAL; minibatches of 1,500
Validation	seed 20001	2,480	selection part 1,466 (racing) · monitoring part 1,014 (never used to choose)
Test	seed 30001	4,500	in-distribution; run once after freezing (guarded by test.done)
Out-of-distribution test	seed 30002	2,900	held-out FFT + Gaussian 1,500 · BIMODAL 1,200 · XL 200
Exact	seed 40001	800	2–10 tasks, solved to proven optimality

Weight-optimisation process

Search on train, race on validation, test once. CMA-ES with random search at equal budget as a control.

Stage 0 *Sobol sensitivity*

1,024 Sobol points + Saltelli N = 256 (3,840 evaluations) on 1,000 training graphs

Track A *fast, no lookahead*

CMA-ES per K ∈ {1, 2, 3, 5}: population 20, $\sigma_0 = 0.3$, one IPOP restart at 40; plus random search

Pre-screen *full training set*

top 15 per run + CMA mean + default weights, rescored on all 7,200

Race *validation*

30 finalists; drop if worse than leader (paired Wilcoxon $p < 0.01$); 25 / 50 / 100% stages

Track B *quality, rollout*

grid L ∈ {3, 5} × depth ∈ {4, 8}; CMA-ES warm-started from A*; race again

Freeze → test *once*

frozen-configs.txt written before the single test run

Objective (minimise), paired against the Original on each instance

$$J = mean\ log(C_i / C_i^{ORIG}) + 0.5 \cdot CVaR_{0.95}\ log(C_i / C_i^{ORIG})$$

Average gain plus a penalty on the worst 5% of instances. 9 unconstrained logits map to two softmax simplices and a logistic λ.

Role of Claude Code

Claude Code (an AI coding assistant) wrote and ran the harness: generator, CMA-ES, Sobol, racing, statistics, reports. It was an engineering tool, not a source of truth. Correctness rests on 201 passing tests, golden traces of the original scheduler, the exact solver and paired statistics. Compute: 29,440 evaluations, ≈ 50 M scheduler runs, ≈ 4.1 h on 12 local threads.

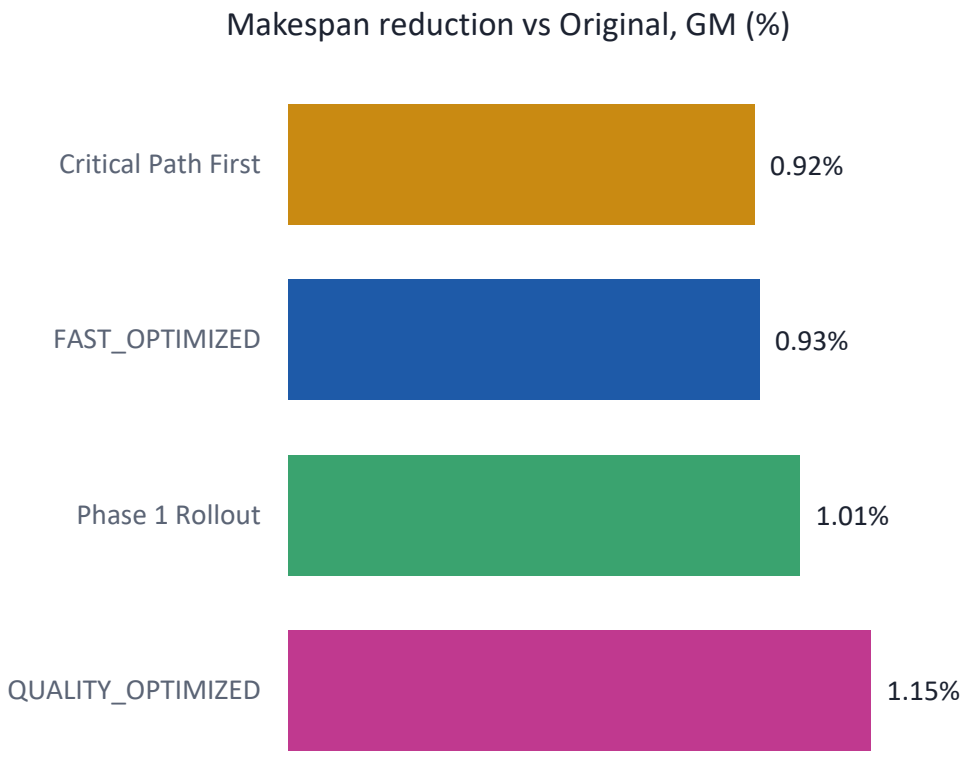
V

Experimental results

Independent test sets, exact optima, robustness and cost.

Comparing scheduling strategies

Phase 2 in-distribution test set: 4,500 instances (seed 30001), run once. Paired against the Original on the same instances.



Reduction = 1 − GM(Cmax / Cmax_Original). Higher is better.

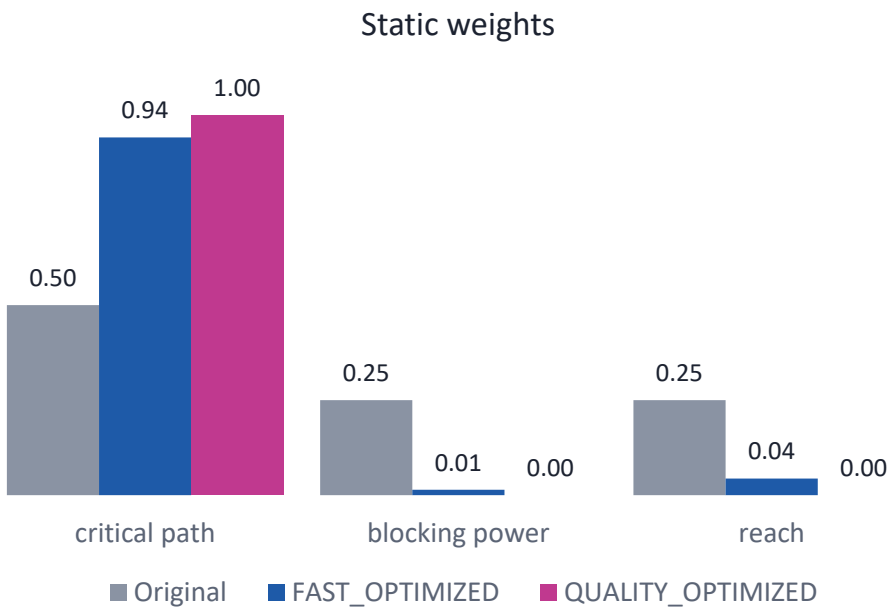
	GM vs Original [95% CI]	mean impr.	W / T / L	worst ratio	GM / LB
Original (DEFAULT)	1.0000	0	—	1.000	1.0443
Critical Path First (batched)	0.9908 [0.9901, 0.9915]	+0.89%	1569 / 2604 / 327	1.084	1.0347
FAST_OPTIMIZED (A*)	0.9907 [0.9901, 0.9914]	+0.90%	1564 / 2620 / 316	1.084	1.0346
Phase 1 Rollout (B0)	0.9899 [0.9893, 0.9906]	+0.98%	1915 / 2542 / 43	1.028	1.0337
QUALITY_OPTIMIZED (B*)	**0.9885 [0.9878, 0.9892]**	**+1.12%**	**1945 / 2495 / 60**	**1.028**	**1.0323**

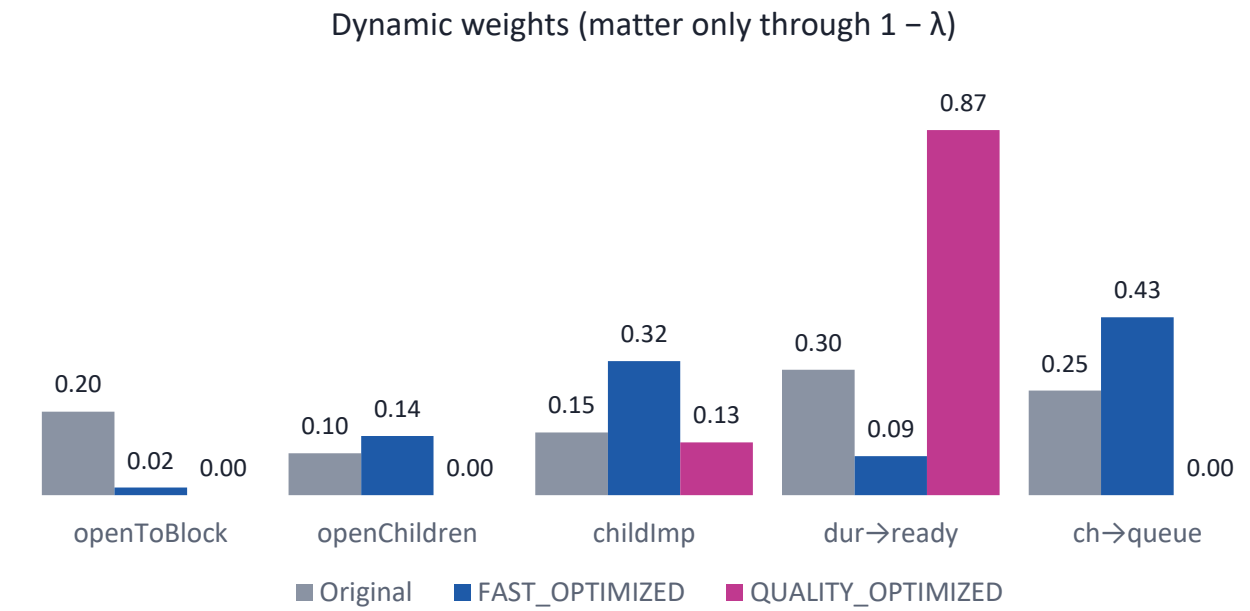
The configurations differ by more than weights

- Original: legacy idle-window lookahead, K = 3, sequential completions.
- FAST: no lookahead, K = 2, batched, reach/max, id tie-break, learned weights.
- Phase 1 Rollout: rollout (depth 8, L = 5, K = 5, contested), batched, original weights.
- QUALITY: the same rollout structure with learned weights.

What weights did optimisation discover?

Every search run converged to critical path only and λ → 1. Dynamic scoring is left as a tie-breaker.





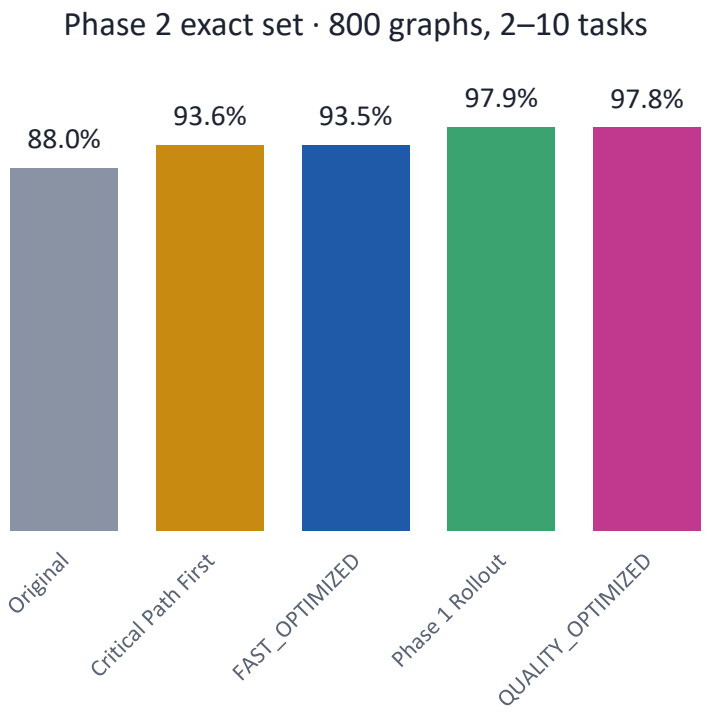
	static (cp, bp, reach)	λ (static share)	dynamic share 1 − λ
Original	0.5, 0.25, 0.25	0.5	0.5
FAST_OPTIMIZED	0.9419505, 0.0143771, 0.0436724	0.9893182	0.0106818
QUALITY_OPTIMIZED	0.9999999971, 2.63e-9, 3.19e-10	0.9989271	0.0010729

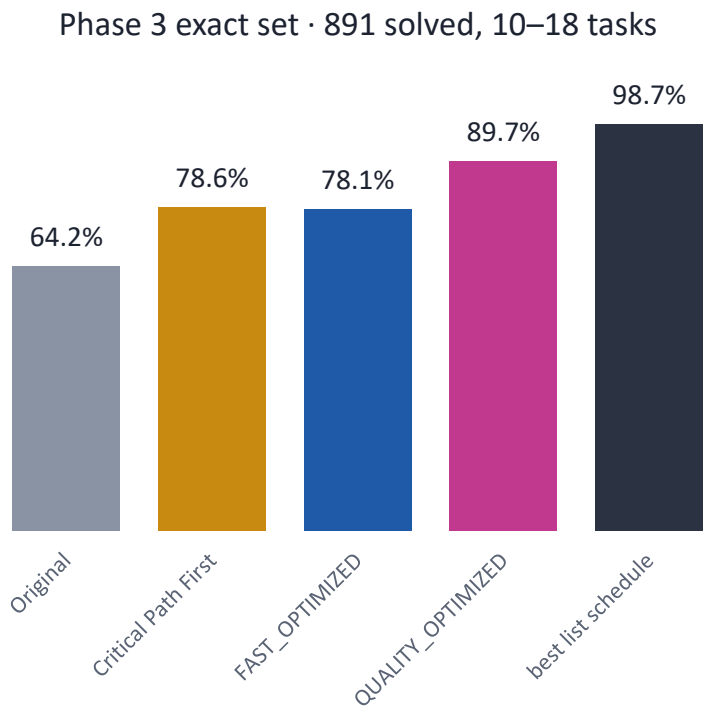
Near zero is not the same as irrelevant

With λ ≈ 0.999 the dynamic term still reorders tasks whose static scores nearly tie. But 30 race finalists with very different dynamic vectors all scored 0.9886–0.9887 on validation, so individual dynamic weights are not identifiable.

How close are we to optimal?

Exact optima on small graphs; lower bounds on large ones. A lower bound is not an optimum.





GM Cmax / OPT	Ph 2 (800)	Ph 3 (891)
Original	1.0091	1.0173
Critical Path First	1.0029	1.0053
FAST_OPTIMIZED	1.0030	1.0054
Phase 1 Rollout	1.0011	—
QUALITY_OPTIMIZED	1.0010	1.0015
best list schedule	—	1.0003

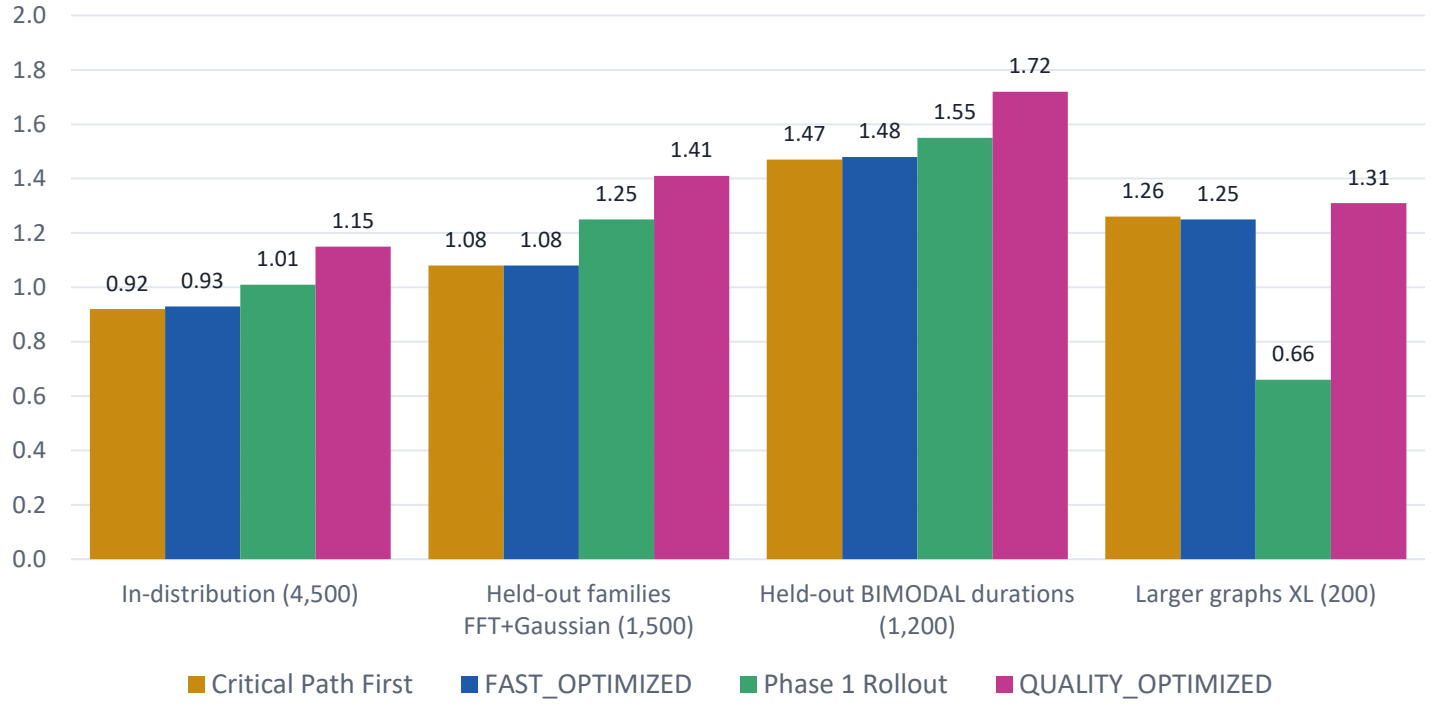
Exact optima allow inserted idle time; “best list schedule” is the best any priority rule can reach.

Large graphs, lower bound only. On the 4,500-instance test set GM Cmax / LB is 1.0443 (Original), 1.0347 (CP-first), 1.0346 (FAST), 1.0337 (Phase 1 Rollout), 1.0323 (QUALITY). On exactly solved graphs the optimum itself averages 1.0280 × LB (Phase 3), so most of the remaining LB gap is slack in the bound, not proven scheduler loss. The true gap on large graphs is unknown.

Robustness and generalisation

The learned configurations were never selected on these sets. QUALITY_OPTIMIZED stays best on every subset.

GM makespan reduction vs Original (%)



Gain shrinks with size vs Phase 1 Rollout (S 0.9999, M 0.9987, L 0.9972 for QUALITY). On XL graphs the original weights in Phase 1 Rollout are weakest.

QUALITY reduction (%) by worker count

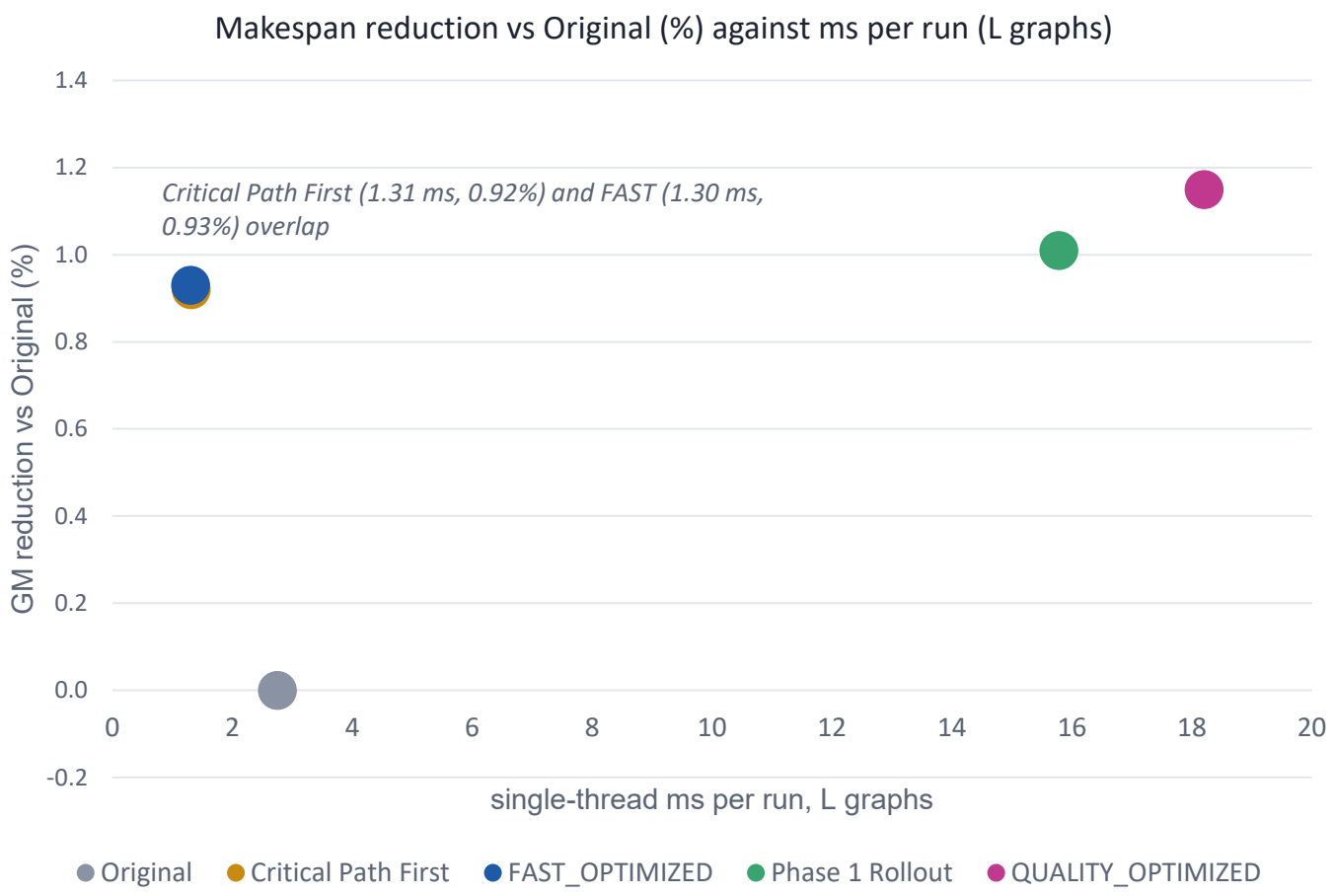


Uncertainty and worst cases

- QUALITY vs Phase 1 Rollout: 0.9986 [0.9984, 0.9988]; OOD 0.9985 / 0.9982 / 0.9934.
- Worst single instance vs Original: QUALITY 1.028 (OOD 1.043); FAST 1.084 (OOD 1.120).
- Losses vs Original: QUALITY 1.33%, FAST 7.0%.

Quality versus computational cost

One timing harness (Phase 2: single thread, 160 instances, median of 3). L-size graphs, 151–600 tasks.



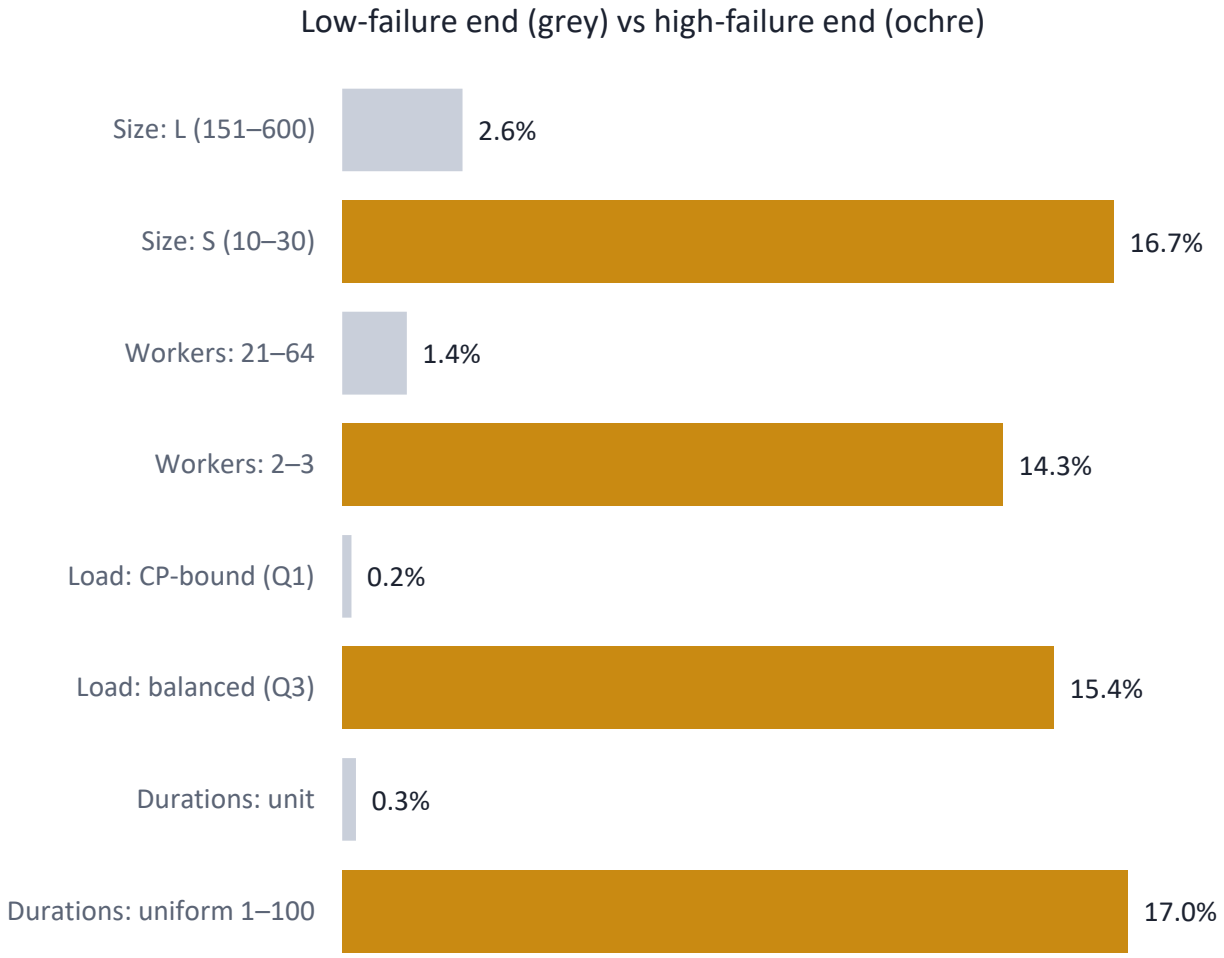
ms per run	S	M	L	XL
Original	0.13	0.53	2.75	4.90
Critical Path First	0.04	0.18	1.31	8.23
FAST_OPTIMIZED	0.05	0.26	1.30	3.01
Phase 1 Rollout	0.63	5.13	15.78	149.5
QUALITY_OPTIMIZED	0.11	2.11	18.20	141.9

Reading the trade-off

- FAST and CP-first: ≈ 0.9% gain at half the Original's time or less.
- QUALITY adds ≈ 0.22 percentage points for 14× FAST's time on L graphs and 47× on XL.
- Phase 3, measured in a separate run of the production code: QUALITY for n ≤ 150, FAST above, keeps 82–90% of QUALITY's gain at FAST's cost on large graphs.

Where do the remaining errors live?

Phase 3: share of instances where Critical Path First is ≥ 1% worse than the best observed schedule (2,700 exploration graphs).



94% of the log gap to the optimum can be closed by changing priorities alone; only 0.03% needs inserted idle time

90% of the best single fixes swap exactly the boundary task, the last one CP-first starts

5–7% of exact CP ties are improvable, vs 18% of decisions on average: the near-tie hypothesis is refuted

0.64 best AUC of any cheap decision-level feature for predicting a bad decision

VI

Conclusions

What we learned, and what the final scheduler delivers.

What did we learn?

Findings supported by the paired tests in the Phase 1–3 reports.

1 **Critical Path First is an exceptionally strong baseline**

0.9908 GM vs Original on test; optimal on 93.6% of tiny and 78.6% of 10–18-task graphs.

2 **Optimisation converges to critical-path dominance**

Every CMA-ES and random-search run reached static ≈ (1, 0, 0) with $\lambda \to 1$.

3 **Most extra components add little under one global weighting**

FAST ≈ CP-first (paired GM 0.9999); dynamic weights sit on a flat plateau.

4 **The improved lookahead adds real quality**

QUALITY 0.9885 vs Original, 0.9977 vs CP-first (Phase 3 final); 97.8% optimal on tiny graphs.

5 **Remaining errors are concentrated**

Small graphs, 2–3 workers, balanced load, non-unit durations; mechanism: worker starvation.

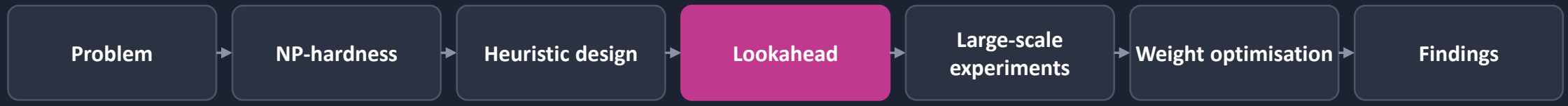
6 **Complexity must pay for itself**

QUALITY is 14–47× FAST's time on L / XL graphs for ≈ 0.2 points; size-gating keeps 82–90% of the gain.

7 **Sophisticated search did not beat simple search**

CMA-ES ≈ random search at equal budget in every group; FAST came from a random-search run.

Final summary



1.15%

GM makespan reduction vs the Original scheduler (QUALITY_OPTIMIZED, 4,500 test graphs)

97.8%

of 800 small graphs scheduled optimally (2–10 tasks)

1.028

worst single-instance ratio vs Original in-distribution; 1.3% of instances lose

The final scheduler: critical-path priority, with a learned near-zero dynamic tie-breaker, plus a depth-8 rollout lookahead on contested decisions.

Principal contribution: a reproducible, statistically controlled evaluation showing that critical path explains almost all of what static and dynamic weighting can achieve. The remaining gains come from simulating the future, concentrated on small, few-worker graphs.

A

Appendix

Definitions, implementation details, supporting tables and caveats.

A1 · Exact component definitions and code locations

component	definition	normalisation	orig. weight	code
critical path cp(v)	$d(v) + \max$ over children of $cp(c)$	÷ overall CP length	0.50	TimeConstraints.criticalPaths
blocking power bp(v)	Σ over children c of $1 / \lvert parents(c) \rvert$	÷ max bp	0.25	TimeConstraints.blockPower
reachable work reach(v)	$\Sigma\, d(u)$ over $\{v\} \cup descendants(v)$	÷ total work (orig.) or ÷ max reach	0.25	TimeConstraints.reachableDuration
openToBlock	$open(v) / \lvert children(v) \rvert$	—	0.20	DynamicScoreLogic.openToBlockRatio
openChildren	$\min(1, open(v) / m)$	m = total workers	0.10	makeDynamicScore
childImportance	$\Sigma\, static(c)$ over open c / $(open(v) \cdot static_max)$	—	0.15	openChildrenImportance
durationToReady	$\alpha\, \hat{d} + (1 - \alpha)(1 - \hat{d})$, $\alpha = \min(1, \lvert R \rvert / m)$	$\hat{d}$ = d / d_max	0.30	durationToRPQSize
childrenToReadyQueue	$(1 - \alpha) \cdot open(v) / maxChildren$	maxChildren = max out-degree	0.25	openChildToRPQSize
blend	$\lambda \cdot static + (1 - \lambda) \cdot dynamic$ over top K	λ = 0.5, K = 3	—	TaskEngine.getNextTask

open(v) counts children whose only remaining unfinished parent is v. |R| is the number of ready tasks at the decision, including the candidates. In the code, maxChildren is held in a variable named maxOpenChild but computed as the maximum number of children of any task.

A2 · Final lookahead: implementation and status

QUALITY_OPTIMIZED (research/archive/…/SchedulerConfig.java)

```
lookaheadMode = ROLLOUT
rolloutDepth = 8   lookaheadCandidates = 5
candidatePoolSize = 5   rolloutIncludesStaticBest = true
contestedDecisionsOnly = true   legacyLoadGate = false
gapCondition = ANY_GAP   completionMode = BATCHED
reachNormalizedByMaximum = true   deterministicTieBreak = true
staticWeights = (0.99999999705, 2.63e-9, 3.19e-10)
dynamicWeights = (1.18e-10, 2.40e-9, 0.12653824, 0.87346176,
1.56e-9)
staticShare λ = 0.99892706
```

Rollout per candidate (DynamicScoreLogic.rolloutMakespan)

```
start candidate at t; other ready tasks wait
loop:
  give free workers ready tasks by static rank
  if nothing runs → return t        (exact)
  if events ≥ depth → return LB(state)
  t ← next completion time
  complete all tasks ending at t, release children
  events ← events + 1
```

Status and honesty notes. All lookahead results in this deck come from the research code in research/archive/phase1-4-research-source (presets PHASE1_ROLLOUT and QUALITY_OPTIMIZED). The tracked src/ tree still holds the original scheduler. An uncommitted port of the rollout into src/main/java/backend/TaskEngine.java was in progress when this deck was built; it chooses candidates slightly differently (3 blended + next by static score) and has not been benchmarked. The original idle-window lookahead was measured to be harmful (Phase 1) and none of its results are attributed to the rollout.

A3 · Phase 1: the original lookahead and the per-change results

MAIN set, 3,600 instances, GM vs Original (lower is better). Selected rows from PHASE1_REPORT.md §6.2.

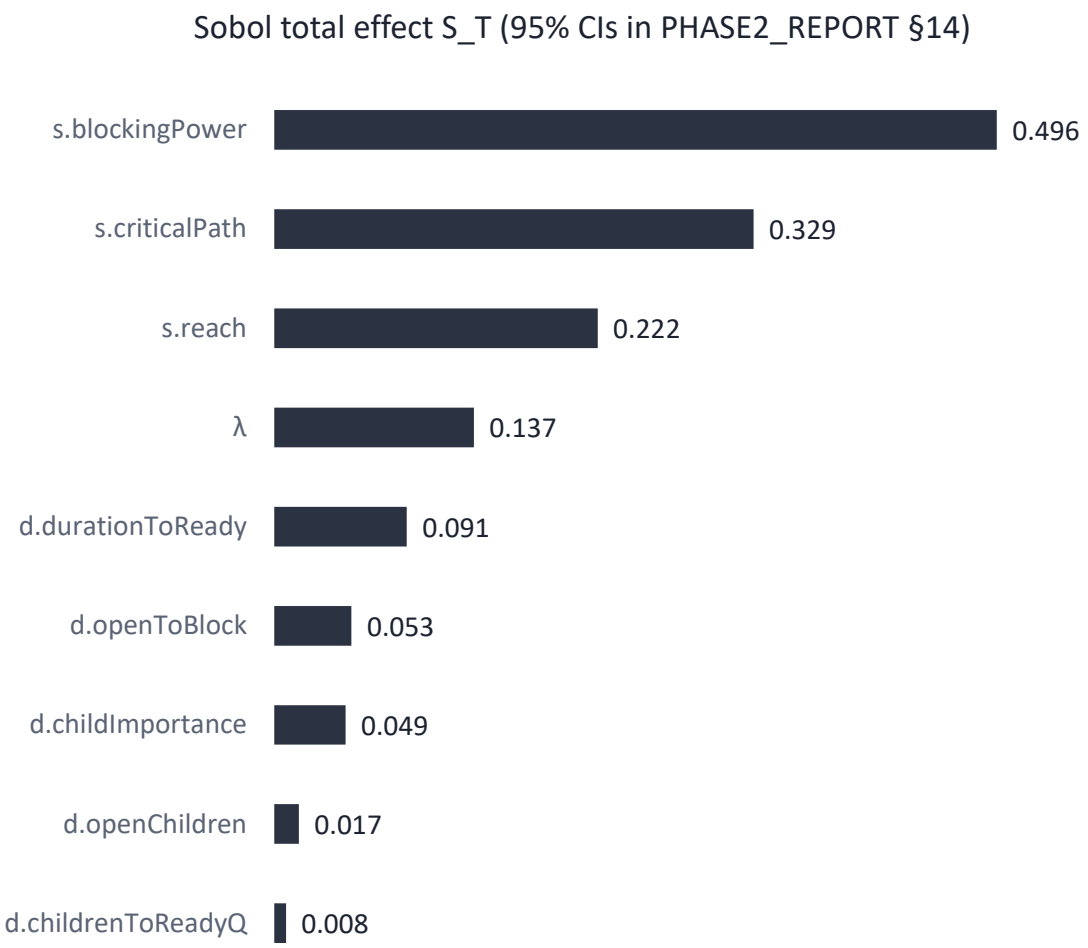
variant	GM vs Original [95% CI]	W / T / L	worst
no lookahead (mode NONE)	0.9971 [0.9966, 0.9975]	556 / 3034 / 10	1.036
static only (K = 1)	0.9939 [0.9933, 0.9946]	1125 / 2117 / 358	1.081
legacy lookahead, gap ignored	1.0064 [1.0057, 1.0071]	63 / 2539 / 998	1.368
legacy lookahead, contested only	1.0119 [1.0108, 1.0130]	73 / 2355 / 1172	1.523
legacy + idle-simulation fix (Change 8)	1.0006 [1.0004, 1.0007]	5 / 3492 / 103	1.138
rollout d = 8, L = 3, contested	0.9900 [0.9893, 0.9907]	1522 / 2022 / 56	1.041
rollout d = 16, L = 5, contested	0.9892 [0.9885, 0.9899]	1606 / 1950 / 44	1.041
rollout full depth, original gates	0.9956	928 / 2535 / 137	1.085
CP-first (sequential)	0.9895	1358 / 2005 / 237	1.250
FIFO / LPT / SPT	1.062 / 1.065 / 1.089	—	1.50–1.67

Takeaways

- The original lookahead never helped: removing it improves the Original by 0.29–0.33%.
- Activating it more often made results worse.
- The rollout with contested activation is what helps.
- Phase 1 recommended configuration on the independent CONFIRM set (2,400): 0.9887 [0.9877, 0.9897], 1,055 / 1,312 / 33.

A4 · Sensitivity analysis and search-method control

Stage 0 (Track A space, K = 3, 1,000 training graphs): Sobol total effects. CMA-ES vs random search at equal budget.



group	CMA-ES: train J; val GM	random: train J; val GM
A-K1	−0.00480; 0.9908	−0.00481; 0.9908
A-K2	−0.00461; 0.9909	−0.00479; 0.9909
A-K3	−0.00479; 0.9909	−0.00505; 0.9912
A-K5	−0.00461; 0.9909	−0.00491; 0.9914
B-L5-d8	−0.01132; 0.9886	−0.01129; 0.9887

Blocking power has the largest effect because giving it weight is harmful: the best region has critical path ≈ 1. A first Stage-0 design was flawed (matrix B was a digital shift of A); it was rebuilt, and the invalid output is kept separately and was not used.

A5 · Phase 3: selective lookahead, final evaluation

GM vs Critical Path First, run once on fresh seeds 81004–81007. Lower is better.

set	n	QUALITY (B*)	full pilot	S1: pilot d8, workers ≤ 8	S2: pilot d8	S4: B* if n ≤ 150, else A*	S4 share of B* gain
In-distribution	2,700	0.9977	0.9973	0.9977	0.9977	0.9980	87%
XL (601–1,500)	120	0.9993	0.9991	0.9994	0.9994	1.0001	0% (by design)
OOD FFT + Gaussian	900	0.9969	0.9965	0.9972	0.9970	0.9974	84%
OOD BIMODAL	600	0.9972	0.9966	0.9971	0.9971	0.9977	82%
OOD Pareto (new)	630	0.9970	0.9962	0.9969	0.9969	0.9973	90%
Exact, GM Cmax/OPT	595 solved	1.0014	1.0010	1.0014	1.0014	1.0014	100%

“Pilot” is an experimental rollout in the Phase 3 study simulator (CP-first continuation), not production code. The full-depth pilot is provably never worse than CP-first and had 0 losses on 4,950 final instances. Study-simulator timings are 25–45× faster than production for the same schedule, so they must not be compared with production timings. Six tie-break rules and a next-completion-aware CP rule stayed within ±0.02% of CP-first.

A6 · Phase 4 (exploratory): weight profiles at bad decisions

Seed 91001: 1,320 instances, 57,840 contested CP-first batches; 17.7% are "bad" (a swap improves the makespan).

profile, applied at one batch	differs from CP	on bad batches: improves / worse	net regret, all batches
Original weights (K = 3)	60.4%	32.8% / 6.6%	−8.3%
FAST weights (K = 2)	28.0%	9.5% / 1.4%	+2.0%
CP + blocking power	53.5%	26.0% / 4.8%	−13.2%
CP + reachable workload	31.7%	12.1% / 1.5%	+3.2%
CP + unlocking	52.6%	25.5% / 5.0%	−1.9%
CP + duration-to-ready	59.2%	33.0% / 7.4%	−40.2%
CP + children-to-ready-queue	19.7%	3.3% / 0.3%	+2.8%

Status

- Exploration only: the development stage log is empty and no configuration was selected or tested.
- A starvation probe (8-event idle fraction) detected bad decisions no better than chance (AUC 0.47).
- An oracle choosing the best profile per batch would capture 62.6% of the best-swap regret, but a cheap depth-2 selector learned nothing (precision 0%).
- These numbers are not used for any claim in the main deck.

A7 · How the examples in this deck were verified

- **Real scheduler code.** Every Gantt chart in Part II–III was produced by running the research version of the scheduler (Controller / TaskEngine) on the instance and reading back each worker's executed tasks.
- **Single-component policies.** Static: weights one-hot, $\lambda = 1$, $K = 1$, no lookahead. Dynamic: $\lambda = 0$, dynamic weights one-hot, K = all ready tasks, no lookahead. All use batched completions and id tie-breaks; dynamic ties fall back to the static order, as in the code.
- **Optimality.** Each counterexample's optimum (idle time allowed) and best list schedule come from the repository's ExactSolver (branch and bound); they coincide on all examples shown.
- **Selection.** Candidate instances came from 30,000 random 4–7-task DAGs (fixed seed). We kept small instances where the decisive choice is strict, not a tie. An independent Python list scheduler matched the Java makespans on all 90,000 static-policy runs.
- **Schedule validity.** Every displayed schedule was checked for durations, precedence, no worker overlap, and a makespan equal to the reported value.
- **Lookahead trace.** Rollout estimates (31 vs 29 at $t = 8$) were printed by a trace hook added to a scratch copy of the code; no repository file was modified.

A8 · Limitations and reproducibility

- Synthetic data from one generator: P | prec | Cmax with known, deterministic durations; no real traces, no duration noise, identical workers.
- Exact optima exist only for ≤ 18 tasks; on large graphs only lower bounds are available, and they are loose.
- QUALITY_OPTIMIZED is 29× the Original's single-thread time on XL graphs (≈ 140 ms per run at n ≈ 1,000).
- Phase 2 search budgets were scaled down from the plan (about 16 h → 4.1 h). Given the plateau and the CMA-ES / random parity, more budget is unlikely to help; that is an inference, not a measurement.
- The rollout continuation (static priority) is not an exact model of execution, so the classical rollout improvement guarantee does not formally hold; measured loss rate vs the Original is 1.3%.
- Timings from different code bases (production vs the Phase 3 study simulator) are never compared directly in this deck.

Reproduce

```
mvn -o test        # 201 tests
mvn -o compile
java -cp target/classes \
  benchmark.ExperimentRunner \
  <main|confirm|xl|exact|…> out 12
java -cp target/classes \
  benchmark.WeightOptimizer \
  <stage> target/phase2/run1 12
java -cp target/classes \
  benchmark.CriticalPathStudy \
  <stage> target/phase3/run1 12
```

Run from the research source tree. Raw outputs: research/raw-results/phase1-experiments, phase2/run1, phase3/run1, phase4/run1.